

Sistema de monitoreo de *Rhynchophorus ferrugineus* en palmeras de Montevideo

Esp. Ing. Bruno Masoller

Carrera de Especialización en Inteligencia Artificial

Director: Ing. Juan Ignacio Cavalieri (FIUBA)

Jurados:

Esp. Ing. Abraham Rodriguez (FIUBA)

Dr. Ing. Álvaro Gomez (FING)

Msc. Ing. Juan Prada (FING)

Ciudad de Montevideo, diciembre de 2025

Resumen

Esta memoria describe la implementación de una plataforma, desarrollada para la Intendencia de Montevideo, que utiliza visión por computadora para detectar la plaga del picudo rojo en su etapa tardía en las palmeras de esta ciudad.

El sistema se desarrolló con el objetivo de reducir costos operativos y mejorar la toma de decisiones para el servicio de arbolado de la institución. Requirió conocimientos de visión por computadora, análisis de datos y aprendizaje profundo, así como de despliegue y operaciones de modelos de aprendizaje de máquinas e infraestructura de soporte.

Índice general

Resumen	I
1. Introducción general	1
1.1. Descripción del problema	1
1.2. Estado del arte	3
1.2.1. Redes convolucionales	3
1.2.2. Detección del picudo rojo en imágenes aéreas	3
1.3. Objetivos y alcance	5
2. Introducción específica	7
2.1. Imágenes generadas por vehículos aéreos	7
2.1.1. Imágenes de drones	7
2.2. Herramientas de soporte en proyectos de AI	8
2.2.1. Gestión de infraestructura	8
2.2.2. Gestión de usuarios y permisos	8
2.2.3. Gestión de etiquetado de datos	9
2.2.4. Gestión de repositorio de datos	9
2.2.5. Visualización de datos	9
2.3. Modelos de visión por computadora	10
2.3.1. Clasificación de objetos	10
2.3.2. Detección de objetos	10
Modelos de dos etapas	10
Modelos de una etapa	11
2.4. Métricas en visión por computadora	11
2.4.1. Precisión, Recuperación y <i>F1-Score</i>	12
2.4.2. <i>Average Precision</i> y <i>Mean Average Precision</i>	13
3. Diseño e implementación	15
3.1. Arquitectura de la plataforma	15
3.2. Despliegue de la infraestructura de soporte	18
3.3. Obtención y gestión de las imágenes	24
3.4. Proceso de etiquetado de datos	26
3.5. Preparación del dataset y estructura del entrenamiento	29
3.6. Desarrollo de la aplicación web	32
4. Ensayos y resultados	37
4.1. Entrenamiento y resultados de modelos	37
4.1.1. Detección de palmeras	39
4.1.2. Detección del picudo rojo	45
4.2. Integración del modelo en la aplicación web	49
5. Conclusiones	53
5.1. Conclusiones generales	53

5.2. Próximos pasos	54
Bibliografía	55

Índice de figuras

1.1. Picudo rojo	1
1.2. Infección y muerte de palmeras por el picudo rojo.	2
1.3. Ortomosaico	2
1.4. Detección de Kagan et al	4
2.1. Arquitectura básica YOLO v11	11
2.2. Matriz de confusión clásica	13
3.1. Diagrama de arquitectura	15
3.2. Diagrama de interfaces externas	18
3.3. Diagrama del sistema	19
3.4. Captura de la página de inicio	23
3.5. Captura del SIG de la IM	24
3.6. Ejemplos de parches	25
3.7. Clases de palmeras	27
3.8. Imagen etiquetada en CVAT.	29
3.9. Recortes de imágenes	32
3.10. Diagrama de caso de uso	33
3.11. Diagrama de secuencia	34
3.12. Captura de inicio de Mis Palmeras	35
4.1. Comparación de performance familia YOLO	37
4.2. Enfoque de <i>transfer learning</i>	38
4.3. Imágenes de entrenamiento	40
4.4. Aumento de datos	41
4.5. Gráficas de entrenamiento del modelo de palmeras	43
4.6. Gráficas de mAP del modelo de palmeras	44
4.7. Matriz de confusión del modelo de palmeras	44
4.8. Gráficas de entrenamiento del modelo del picudo rojo	47
4.9. Gráficas de mAP del modelo del picudo rojo	48
4.10. Matriz de confusión del modelo del picudo rojo	48
4.11. Captura carga de datos	49
4.12. Ejemplo de predicción	50

Índice de tablas

1.1. Comparativa de estudios	5
3.1. Tecnologías utilizadas	20
3.2. Proceso de etiquetado	28
3.3. Jornadas de etiquetado	29
3.4. Operaciones de procesamiento	31
4.1. Hardware de entrenamiento	39
4.2. Transformaciones del dataset de palmeras	40
4.3. Experimentos modelo detección de palmeras	42
4.4. Transformaciones del dataset del picudo rojo	45
4.5. Experimentos modelo detección del picudo rojo	46
4.6. Posprocesamiento de predicciones	51

Capítulo 1

Introducción general

En este capítulo se presenta una introducción general al trabajo realizado. Se describe el problema que el *Rhynchophorus ferrugineus* (picudo rojo) presenta en las palmeras de la ciudad de Montevideo, el estado del arte en cuanto a trabajos similares y los objetivos planteados por la Intendencia de Montevideo (IM) y por el equipo de trabajo.

1.1. Descripción del problema

El picudo rojo, que puede observarse en la figura 1.1, es un insecto que afecta a las palmeras, especialmente a la *Phoenix Canariensis*, que es la especie más común en Montevideo. Este insecto ha causado un daño significativo en la flora de la ciudad, lo que ha llevado a la IM a enfocarse en su control y erradicación.



FIGURA 1.1. Picudo rojo ¹.

Este escarabajo supone una amenaza grave para las palmeras infectadas puesto que las larvas de este insecto se alimentan del tejido interno de la planta, causando su colapso estructural en un período de meses (dado la gran cantidad de larvas que genera cada hembra [1]), como puede verse en la figura 1.2a. Esta amenaza aumenta según la época del año, puesto que el insecto tiene diferentes tasas de dispersión y reproducción dependiendo de la temperatura y la humedad. Existen varios tipos de picudos [2], algunos autóctonos y otros introducidos.

La plaga del picudo rojo llegó a Uruguay en 2022 [3] y se esparció rápidamente por la ciudad de Montevideo. De las 25 000 palmeras que forman una parte esencial de la ciudad, muchas ya han sucumbido a la plaga, donde se estima que para el año 2030 el ecosistema se verá ampliamente afectado [4] si no se toman

¹Imagen tomada de https://es.wikipedia.org/wiki/Rhynchophorus_ferrugineus

medidas de control adecuadas. Últimamente también se ha esparcido por el interior del país, como puede observarse en la figura 1.2b que pertenece a la ruta 5 de Uruguay, donde se han encontrado palmeras muertas por la plaga.



(A) Palmera infectada por el picudo rojo.



(B) Palmeras muertas en la ruta 5 de Uruguay.

FIGURA 1.2. Ejemplos de infección y muerte de palmeras por el picudo rojo detectados en Uruguay.

Entre los métodos de control fitosanitarios que se utilizan para combatir la plaga se encuentran la endoterapia [5], el baño, la cirugía [6] y finalmente la remoción. Sin embargo, estos métodos son costosos y requieren de un monitoreo constante para detectar la presencia del picudo rojo. Para la IM no es solamente una cuestión ecológica sino también económica. En este sentido, el servicio de arbolado realiza un seguimiento de las palmeras afectadas por la plaga. Para ello, se registra su ubicación y estado de salud mediante campañas de detecciones a pie. Este proceso manual requiere de mucho tiempo y recursos humanos, por lo que resulta imprescindible un sistema automatizado que permita detectar la presencia de la plaga en lugares específicos de Montevideo.

Uno de los recursos aún no explotados por la IM para el monitoreo de la plaga es el uso de imágenes aéreas obtenidas mediante drones, disponibles por el servicio de geomática de esta institución. Estos ortomosaicos, que pueden observarse en la figura 1.3, permiten obtener información detallada sobre el estado de las palmeras y su entorno.



FIGURA 1.3. Ejemplo de ortomosaico tomado del parque José Enrique Rodó de Montevideo ².

El análisis de estas imágenes es un proceso complejo que requiere de técnicas avanzadas de procesamiento de imágenes y aprendizaje automático, así como también herramientas que brinden soporte a estas actividades.

1.2. Estado del arte

En los últimos años, el campo de la visión por computadora (CV, por su sigla en inglés de *Computer Vision*) ha experimentado un crecimiento exponencial, impulsado principalmente por los avances en el aprendizaje profundo y las redes neuronales convolucionales (CNN, por su sigla en inglés de *Convolutional Neural Networks*). Estas tecnologías han permitido el desarrollo de soluciones innovadoras para tareas de detección, clasificación y segmentación en imágenes, lo que ha abierto nuevas posibilidades para aplicaciones en diversos ámbitos, tales como la monitorización ambiental y la agricultura de precisión. La detección de plagas mediante imágenes aéreas, en particular la del picudo rojo, ha cobrado relevancia debido al creciente impacto que este insecto tiene en la salud de las palmeras urbanas, principalmente en países del Mediterráneo y en el Medio Oriente [2]. En esta sección se revisan las principales técnicas y enfoques que han marcado el estado del arte en la detección de plagas a partir de imágenes aéreas. Se aborda tanto arquitecturas tradicionales como las más recientes, donde se destacan sus fortalezas, limitaciones y la evolución de sus aplicaciones en escenarios reales.

1.2.1. Redes convolucionales

La visión por computadora abarca una amplia gama de tareas, tales como la detección de objetos, la segmentación de imágenes, el reconocimiento de patrones y la clasificación de imágenes. Estas tareas suelen abordarse mediante modelos de CNN. Tendencias recientes evidencian avances significativos en el diseño de estas redes para el procesamiento de imágenes. En particular, se ha puesto énfasis en el tratamiento de imágenes de alta resolución capturadas mediante vehículos aéreos no tripulados (UAV, por su sigla en inglés de *Unmanned Aerial Vehicles*), lo que representa un área de aplicación de creciente interés del sector académico y privado [7] [8].

1.2.2. Detección del picudo rojo en imágenes aéreas

La relevancia de la detección de la plaga del picudo rojo a nivel global ha propiciado el surgimiento de diversas líneas de investigación. En este sentido, se han desarrollado enfoques basados en CV a partir de imágenes aéreas, haciendo especial uso de arquitecturas de CNN tales como YOLO [9].

Entre los estudios analizados, la implementación de la detección mediante imágenes aéreas de forma exclusiva representa una metodología relativamente novedosa. En este contexto, en el estudio *Automatic large scale detection of red palm weevil infestation using street view images* [10], se emplea un enfoque combinado. Inicialmente, se localizan las palmeras a través de una CNN (Faster R-CNN [11]) aplicada a imágenes aéreas. Posteriormente, utilizando la misma arquitectura, se extrae la corona de la palmera a partir de imágenes capturadas en *street view*, lo

²Imagen tomada del sistema de información geográfica de la IM: <https://sig.montevideo.gub.uy/>

que permite clasificar a la planta en función de la presencia o ausencia de infección. Un resumen de este trabajo se presenta en la figura 1.4.

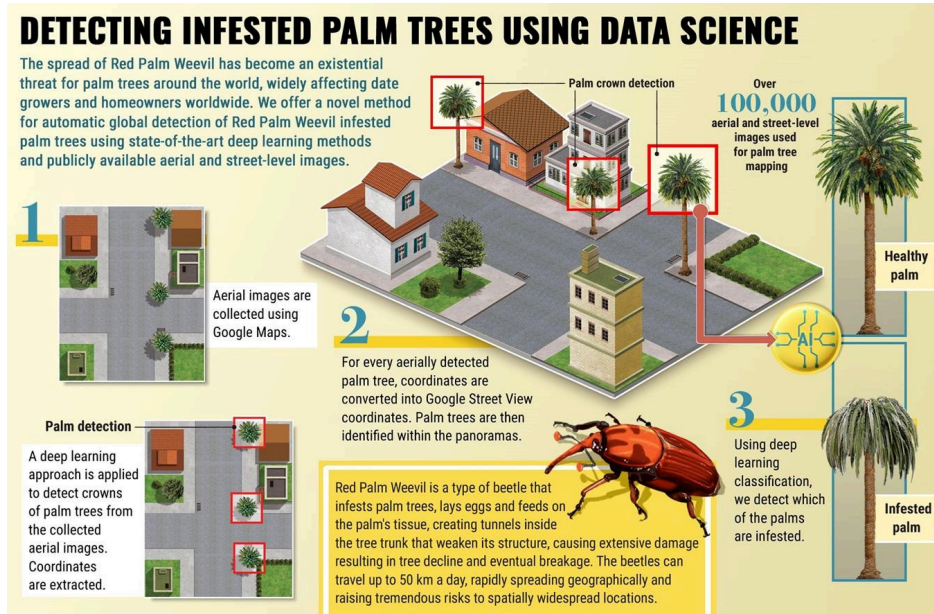


FIGURA 1.4. Proceso de detección realizado por Kagan et al, utilizando Street View ³.

Por otro lado, la detección de palmeras sí es ampliamente estudiada en la literatura, siendo un tema recurrente en la investigación de CV. Se han desarrollado diversos estudios. En *Implementation of slicing aided hyper inference (SAHI) in YOLOv8 to counting oil palm trees using high-resolution aerial imagery data* [12] se presenta un enfoque que utiliza la arquitectura YOLOv8 para detectar palmeras en imágenes aéreas de alta resolución. Además, utiliza *Slicing Aided Hyper Inference* para mejorar la precisión de la detección. Este enfoque se basa en la idea de dividir las imágenes en regiones más pequeñas y realizar inferencias en cada una de ellas, lo que permite una detección más precisa de los objetos de interés, algo esencial en el caso de objetos pequeños como las palmeras. Para el estudio, se utilizaron imágenes RGB capturadas con un dron Trinity F90+ a 200 metros de altura y una implementación de segmentación (SAHI) de 3000×3000 px. Si bien el estudio se centra en la detección de palmeras de aceite, su metodología puede ser adaptada para la detección del picudo rojo. La tabla 1.1 resume los métodos utilizados en la detección de palmeras en diferentes estudios anteriores.

³Imagen obtenida de: <http://arxiv.org/abs/1506.01497>

TABLA 1.1. Comparación entre diferentes estudios de detección de palmeras ⁴.

Autor y año	Método	Evaluación
Mukhles Sir Monea et al, 2022 [13]	YOLOv3	5,76 % (MAPE)
Hery Wibowo et al, 2022 [14]	YOLOv3, YOLOv4, YOLOv5m	97,28 % (v3), 97,74 % (v4), 94,94 % (v5m). (F1-Score)
Adel Ammar et al, 2022 [15]	Faster R-CNN	94,99 % (Precision), 84 % (Recall), 83 % (AP IoU)
Wardana et al, 2023 [16]	YOLOv8	98,50 % (Overall Accuracy)
Deta Sandya Prasitha et al, 2022 [17]	YOLOv5s, YOLOv5m, YOLOv5l, YOLOv5x	0,82 (v5s), 0,84 (v5m), 0,85 (v5l), 0,86 (v5x). (Average F1-Score)
Nuwar et al, 2022 [18]	YOLOv5	0,895 (F1-Score)
Junos et al, 2022 [19]	YOLOv3n	97,20 % (mAP), 0,91 (F1-Score)

También, se han realizado estudios controlados como en *Red Palm Weevil Detection in Date Palm Using Temporal UAV Imagery* [20], en donde se concluye que la detección del picudo rojo es posible mediante imágenes aéreas en el rango infrarrojo.

1.3. Objetivos y alcance

La finalidad principal del presente trabajo consistió en validar una prueba de concepto para la detección tardía del picudo rojo en palmeras de Montevideo, utilizando imágenes aéreas capturadas mediante drones. Con este objetivo, se desarrolló un sistema fundamentado en técnicas de CV y aprendizaje profundo, que permiten identificar la presencia de la plaga. Para ello, se utilizó información centralizada en el sistema de información geográfica (SIG) de la IM [21], complementado con herramientas adicionales que facilitaron la generación del conjunto de datos de entrenamiento.

Adicionalmente, se planteó ante la IM la necesidad de optimizar los procesos de detección de la plaga, con miras a una mejora en la asignación de recursos humanos y económicos. El alcance del trabajo se centró en el desarrollo de una plataforma que permite a los usuarios del sector de áreas verdes de la IM realizar un seguimiento de la expansión de la plaga mediante el monitoreo del estado de salud de las palmeras. En los capítulos siguientes se detalla tanto la descripción de las herramientas utilizadas como el proceso metodológico implementado.

⁴Tabla obtenida de *Implementation of slicing aided hyper inference (SAHI) in YOLOv8 to counting oil palm trees using high-resolution aerial imagery data* [12]

Capítulo 2

Introducción específica

En este capítulo se presentan en detalle las tecnologías y conceptos utilizados para el trabajo. Se parte de una introducción a las imágenes generadas por vehículos aéreos, especialmente aquellas generadas por UAV. Luego, se mencionan herramientas adicionales utilizadas en los procesos de soporte de proyectos orientados a AI (por su sigla en inglés de *Artificial Intelligence*), como lo son el proceso de etiquetado y la gestión de los datos. Finalmente, se puntualiza en modelos y métricas utilizados en CV para este tipo de trabajos.

2.1. Imágenes generadas por vehículos aéreos

Los UAV se han consolidado como herramientas esenciales para la obtención de imágenes aéreas de alta resolución. Esto favorece avances en áreas como la agricultura de precisión, la vigilancia ambiental y la inspección de infraestructuras [22]. La flexibilidad operativa de estos drones permite planificar vuelos a medida para cada proyecto específico, lo que facilita el acceso a zonas amplias o de difícil acceso. Además, la incorporación de equipamientos especializados, como cámaras multispectrales, sensores térmicos y sistemas LiDAR, posibilita la captura de datos en diferentes bandas del espectro electromagnético, fundamentales para un análisis detallado y preciso [23].

En contraste con los métodos tradicionales y los vuelos tripulados (que cubren áreas mayores pero implican costos y logísticas más complejas [24]), los UAV ofrecen eficiencia en tiempo y costos. Asimismo, reducen la exposición del personal a ambientes potencialmente peligrosos. La adaptación a condiciones controladas y la capacidad de replicar vuelos contribuyen a la generación de series temporales que facilitan el monitoreo y análisis evolutivo de fenómenos específicos, como la plaga del picudo rojo.

2.1.1. Imágenes de drones

Las imágenes generadas por drones se destacan por su alta resolución y capacidad para capturar detalles finos del terreno. Estas imágenes suelen tener una mayor resolución (medida en cm/px) en comparación con las generadas por aviones. El DJI Phantom 4 PRO [25] es ampliamente utilizado en aplicaciones de mapeo y fotogrametría debido a su capacidad para generar ortomosaicos con una resolución de hasta 5 cm/px (volando a 100 metros de altura). Otro dron ampliamente utilizado es el DJI MAVIC 3 Enterprise RTK [26]. La integración de tecnología RTK [27] en este dron permite una mayor precisión en la georreferenciación de

las imágenes capturadas, a diferencia del DJI Phantom 4 PRO, que necesita puntos de apoyo relevados en el campo tomados con un GPS RTK (en las ejecuciones de vuelos con el DJI Phantom de la IM, los puntos de apoyo fueron tomados con GNSS SinoGNSS T300 Plus [28], utilizando corrección en tiempo real por NTRIP [29] con la base UYMO del Instituto Geográfico Militar uruguayo).

2.2. Herramientas de soporte en proyectos de AI

La cara visible de los proyectos de inteligencia artificial se fundamenta, en general, en los resultados y aplicaciones derivados de modelos de aprendizaje automático. No obstante, al igual que en cualquier proyecto de software, resulta fundamental integrar buenas prácticas de ingeniería de software, lo que implica focalizarse en atributos específicos del sistema, tales como la escalabilidad, la seguridad y la recuperabilidad, entre otros [30]. Para alcanzar estos objetivos, es crucial considerar disciplinas que faciliten su consecución, como la gestión de la configuración o la gestión de datos. A continuación, se exponen algunas herramientas de apoyo a varios de estos procesos.

2.2.1. Gestión de infraestructura

La correcta administración de la infraestructura es esencial para el despliegue y mantenimiento de aplicaciones de inteligencia artificial, tanto en entornos de desarrollo como en producción. Para ello, en la industria se suelen utilizar diversas herramientas, entre las que se encuentran:

- Docker y Docker Compose [31] [32]: estas herramientas permiten la creación de contenedores que encapsulan una aplicación (o varias aplicaciones) y sus dependencias, lo que asegura un entorno de ejecución uniforme y reproducible en diferentes sistemas. Docker Compose facilita la orquestación de múltiples contenedores y posibilita la simulación de entornos complejos en un ambiente local.
- Vagrant [33] [34]: se utiliza para la configuración y despliegue de máquinas virtuales. Esta herramienta ofrece un entorno virtualizado consistente y de fácil replicación, lo que resulta útil para la estandarización del entorno de desarrollo simplemente con un archivo de configuración (Vagrantfile).

2.2.2. Gestión de usuarios y permisos

La seguridad y la privacidad son aspectos críticos en el desarrollo de aplicaciones de inteligencia artificial. La correcta gestión de usuarios y permisos es esencial para garantizar que solo las personas autorizadas tengan acceso a datos sensibles y funcionalidades específicas del sistema. Para ello, se pueden implementar herramientas compatibles con el protocolo *Lightweight Directory Access Protocol* (LDAP) [35], como lo es *Light LDAP* (LLDAP) [36]. Esta herramienta permite la autenticación y autorización de usuarios, lo que simplifica la gestión centralizada de credenciales y permisos. Además, se pueden establecer políticas de acceso basadas en roles, y así contribuir a una mayor seguridad y control sobre el sistema.

Actualmente, Keycloak [37] es una de las herramientas más utilizadas para la gestión de identidad y acceso. Proporciona funcionalidades como inicio de sesión único (SSO), autenticación multifactor (MFA) y federación de identidades, lo

que facilita la integración con diversas aplicaciones y servicios. Keycloak es compatible con protocolos estándar como OAuth2, OpenID Connect y SAML, lo que permite una gestión segura y eficiente de usuarios y permisos en entornos de AI. También se integra fácilmente con LDAP, lo que permite aprovechar infraestructuras de autenticación ya existentes.

2.2.3. Gestión de etiquetado de datos

La calidad de los modelos de CV dependen en gran medida de la precisión y consistencia del etiquetado de datos. Para optimizar este proceso, existen herramientas especializadas como CVAT [38], que facilitan la anotación de conjuntos de datos para el entrenamiento y validación de modelos. Su interfaz intuitiva y funcionalidades colaborativas permiten una asignación precisa de etiquetas en imágenes, lo que reduce errores y mejora la calidad de los datos. Entre las funcionalidades de CVAT se encuentra la integración con protocolos LDAP y la gestión de datos utilizando *object storage*.

FiftyOne [39], otra herramienta ampliamente utilizada en la industria, permite visualizar, analizar y gestionar los conjuntos de datos etiquetados. Facilita la identificación de inconsistencias y errores en las anotaciones, lo que resulta crucial para la mejora continua de los modelos de aprendizaje. Esta herramienta se destaca por su capacidad para manejar grandes volúmenes de datos y su integración con frameworks de aprendizaje automático populares, lo que la convierte en una opción versátil para proyectos de CV.

2.2.4. Gestión de repositorio de datos

El manejo eficiente de grandes volúmenes de datos es un aspecto indispensable en los proyectos de AI, especialmente las imágenes en proyectos de CV. Entre las opciones *open source* se encuentran:

- MinIO [40]: solución de almacenamiento de objetos compatible con S3 [41] (*S3-compliant*). Permite gestionar grandes volúmenes de datos de manera escalable y segura. Su implementación posibilita la integración tanto en entornos de almacenamiento en la nube como en sistemas locales, lo que asegura la disponibilidad continua de la información. Se integra fácilmente con herramientas de anotaciones como CVAT.
- MongoDB [42]: base de datos NoSQL que se utiliza para almacenar metadatos y gestionar información asociada a los conjuntos de datos. La flexibilidad y la escalabilidad de Mongo facilitan la administración de datos estructurados y no estructurados, lo que mejora la eficiencia al realizar consultas y análisis detallados.

La integración de estas herramientas de soporte establecen un flujo de trabajo sistematizado y robusto. Este enfoque integral asegura la reproducibilidad de los resultados, optimiza la gestión de recursos y facilita tanto el mantenimiento como la evolución del sistema a lo largo del tiempo.

2.2.5. Visualización de datos

La visualización de datos es una herramienta fundamental para interpretar y comunicar los resultados obtenidos en proyectos de inteligencia artificial, así como

también para el monitoreo e interacción con los usuarios finales. En este contexto, existen diversas herramientas que facilitan esta tarea, entre las que se destacan:

- Angular [43]: framework de desarrollo web que permite crear interfaces de usuario interactivas y dinámicas. Su arquitectura basada en componentes facilita la construcción de aplicaciones modulares y escalables, lo que es ideal para proyectos que requieren una visualización compleja de datos, como mapas interactivos.
- FastAPI [44]: framework web moderno y de alto rendimiento para construir APIs con Python. Su diseño asíncrono y su facilidad de uso permiten desarrollar servicios backend eficientes que pueden manejar grandes volúmenes de solicitudes, lo que es crucial para aplicaciones que requieren acceso rápido a datos y modelos de AI.

2.3. Modelos de visión por computadora

El desarrollo de los modelos de CV ha permitido avances notables en la automatización y análisis de imágenes. Entre las tareas de clasificación y detección de objetos, las CNN se destacan con altos niveles de precisión y eficiencia.

2.3.1. Clasificación de objetos

La tarea de clasificación consiste en asignar una etiqueta o categoría a una imagen (o a regiones específicas). Entre los modelos más utilizados en esta área se encuentra la familia EfficientNet. Esta familia fue introducida en 2019 por investigadores de Google en el artículo *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks* [45]. Este conjunto de arquitecturas se caracteriza por su innovador enfoque de escalado compuesto, que equilibra de forma simultánea la profundidad, el ancho y la resolución de la red para así lograr mejores resultados que los enfoques tradicionales.

2.3.2. Detección de objetos

La detección de objetos se enfoca no solo en clasificar, sino también en localizar de forma precisa los elementos de interés dentro de una imagen. Esta tarea se ha abordado mediante dos grandes paradigmas: los modelos de dos etapas y los modelos de una etapa.

Modelos de dos etapas

Los modelos de dos etapas, representados de manera destacada por Faster R-CNN, se caracterizan por un proceso secuencial en el que se generan propuestas de regiones de interés (ROI, del inglés *Region of Interest*) para luego clasificarlas y refinar sus límites. La primera etapa utiliza una red de propuestas de regiones (RPN, por su sigla en inglés de *Region Proposal Network*) que sugiere ubicaciones potenciales de objetos. En la segunda etapa, estas propuestas se analizan en detalle mediante un clasificador que asigna una etiqueta y ajusta la caja delimitadora. Esta aproximación, aunque computacionalmente más costosa, ofrece una alta precisión en la detección, lo que la hace adecuada para aplicaciones donde la exactitud es prioritaria.

Modelos de una etapa

En contraste, los modelos de una etapa, como YOLO, abordan el problema de la detección en un único paso. Estos modelos dividen la imagen en una cuadrícula y, de forma simultánea, predicen la probabilidad de presencia de un objeto y la localización de su correspondiente caja delimitadora. Esta integración de procesos permite una mayor velocidad de inferencia, lo que hace de YOLO una opción idónea para aplicaciones en tiempo real, aunque hay antecedentes del uso de este tipo de modelos para la detección de objetos con una alta eficiencia en la sección 1.2. YOLOv11 [46] es una de las últimas versiones de la familia de modelos YOLO, desarrollada por Ultralytics. Esta versión, que se puede observar en la figura 2.1, introduce mejoras significativas en términos de precisión y velocidad, lo que optimiza tanto la arquitectura del modelo como los algoritmos de entrenamiento.

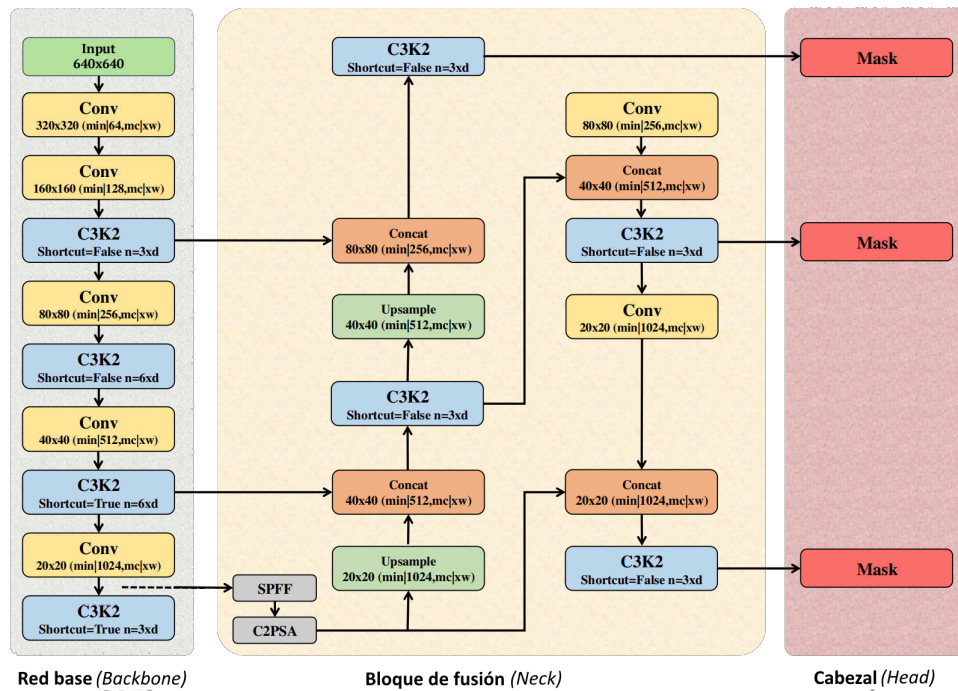


FIGURA 2.1. Arquitectura de YoloV11 para el caso de segmentación, análogo a la tarea de detección, en donde se cambian los cabezales de segmentación por cabezales de detección¹.

2.4. Métricas en visión por computadora

La evaluación cuantitativa de los modelos de CV es esencial para determinar su desempeño y compararlos de manera objetiva. En este contexto, se utilizan diversas métricas que permiten analizar tanto la precisión en la clasificación de imágenes como la efectividad en la detección y localización de objetos. A continuación, se describen las métricas más utilizadas en el campo.

¹Imagen obtenida y adaptada de <https://www.researchsquare.com/article/rs-5582314/v1>

2.4.1. Precisión, Recuperación y *F1-Score*

Para evaluar el rendimiento de los modelos de visión por computadora, es fundamental comprender ciertos términos básicos que se utilizan en el cálculo de las métricas. En este contexto, los verdaderos positivos (TP, del inglés *True Positives*) corresponden a los casos en los que el modelo identifica correctamente la presencia de un objeto de interés. Los falsos positivos (FP, del inglés *False Positives*) se refiere a las instancias en las que el modelo indica la presencia de un objeto de interés cuando en realidad este no está presente. Los falsos negativos (FN, del inglés *False Negatives*) y verdaderos negativos (TN, del inglés *True Negatives*) son términos que describen situaciones opuestas: los FN representan los casos en los que el modelo no detecta un objeto de interés que sí existe en la imagen, mientras que los TN indican las ocasiones en las que el modelo reconoce correctamente la ausencia de un objeto de interés. A partir de estos conceptos, se definen las siguientes métricas clave:

- **Precisión (*Precision*)**: mide la proporción de verdaderos positivos sobre el total de predicciones positivas realizadas por el modelo. Una alta precisión indica que, de todos los objetos identificados, la mayoría corresponde efectivamente a objetos de interés. Se define como:

$$\text{Precisión} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2.1)$$

- **Recuperación (*Recall*)**: representa la proporción de verdaderos positivos detectados sobre el total de objetos de interés presentes en el conjunto de datos. Un valor elevado de *recall* sugiere que el modelo es capaz de capturar la mayoría de los objetos existentes. Se define como:

$$\text{Recuperación} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (2.2)$$

- **Exactitud (*Accuracy*)**: mide la proporción de predicciones correctas (tanto verdaderos positivos como verdaderos negativos) sobre el total de predicciones realizadas. Esta métrica es útil en contextos donde las clases están equilibradas, pero puede ser engañosa en conjuntos de datos desbalanceados. Se define como:

$$\text{Exactitud} = \frac{\text{TP} + \text{TN}}{\text{Total de predicciones}} \quad (2.3)$$

- ***F1-Score***: es la media armónica entre *precision* y *recall*, y proporciona una medida equilibrada cuando se requiere considerar ambas métricas simultáneamente. Se define como:

$$F1 = 2 \cdot \frac{\text{Precisión} \cdot \text{Recuperación}}{\text{Precisión} + \text{Recuperación}} \quad (2.4)$$

Un resumen intuitivo de estas métricas se puede visualizar en la imagen [2.2](#).

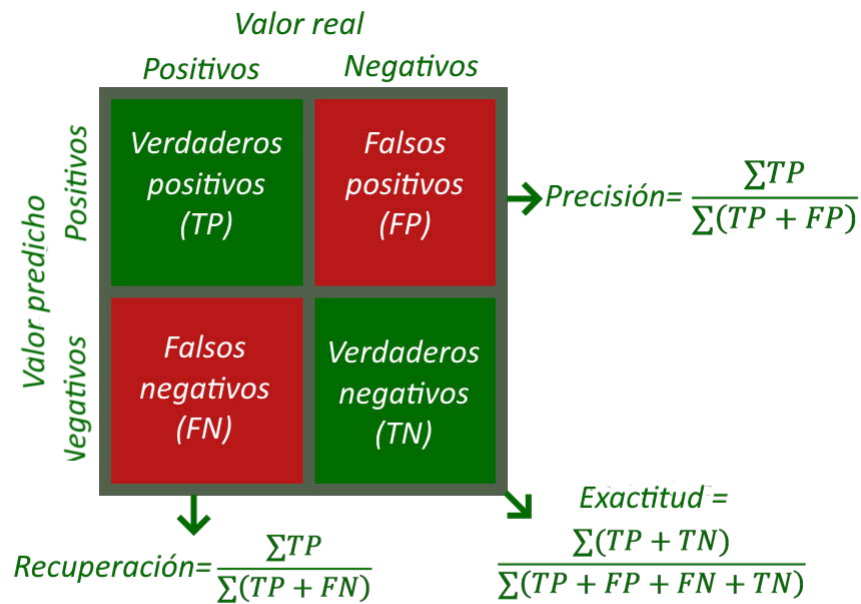


FIGURA 2.2. Cálculo de precisión, recuperación y exactitud a partir de la matriz de confusión.

2.4.2. Average Precision y Mean Average Precision

La evaluación del desempeño en tareas de detección de objetos requiere considerar cómo varía la precisión del modelo ante distintos umbrales de decisión. Se suele utilizar dos métricas claves: la precisión promedio (AP, del inglés *Average Precision*) y media de la precisión promedio (mAP, del inglés *Mean Average Precision*).

- AP: para cada clase de objeto, se calcula el promedio de la precisión, integrando la curva de precisión-recuperación (ROC, por su sigla en inglés de *Receiver Operating Characteristic*) [47]. La AP resume el rendimiento del detector en diferentes umbrales de decisión.
- mAP: es la media aritmética de la AP calculada para cada clase. El mAP es ampliamente utilizado en competencias y evaluaciones de modelos de detección de objetos, ya que proporciona una única medida que resume la efectividad global del modelo.

La correcta selección y combinación de estas métricas permite una evaluación integral de los modelos de CV, lo que facilita la identificación de áreas de mejora así como la comparación objetiva entre diferentes enfoques o arquitecturas implementadas.

Capítulo 3

Diseño e implementación

En este capítulo se describen los aspectos más relevantes del diseño e implementación de la plataforma, así como el procesamiento de los datos. Inicialmente se presenta la arquitectura y se describen los componentes e interacciones. Luego se describe el proceso de despliegue de la infraestructura de soporte, tanto en el ambiente de desarrollo como en el de producción. Posteriormente se enfatiza en tarea de etiquetado de datos, el proceso de preparación del dataset y la estructura para el entrenamiento. Finalmente se presenta el desarrollo y despliegue de la aplicación web junto con los modelos de detección.

3.1. Arquitectura de la plataforma

El objetivo principal de este trabajo fue brindar un producto para el servicio de arbolado, en donde se les permita consultar el estado de la plaga en base a imágenes aéreas. Para ello, se diseñó y desarrolló la plataforma de la figura 3.1, que integra varios módulos con el fin de soportar todos los procesos relacionados con el desarrollo del software.

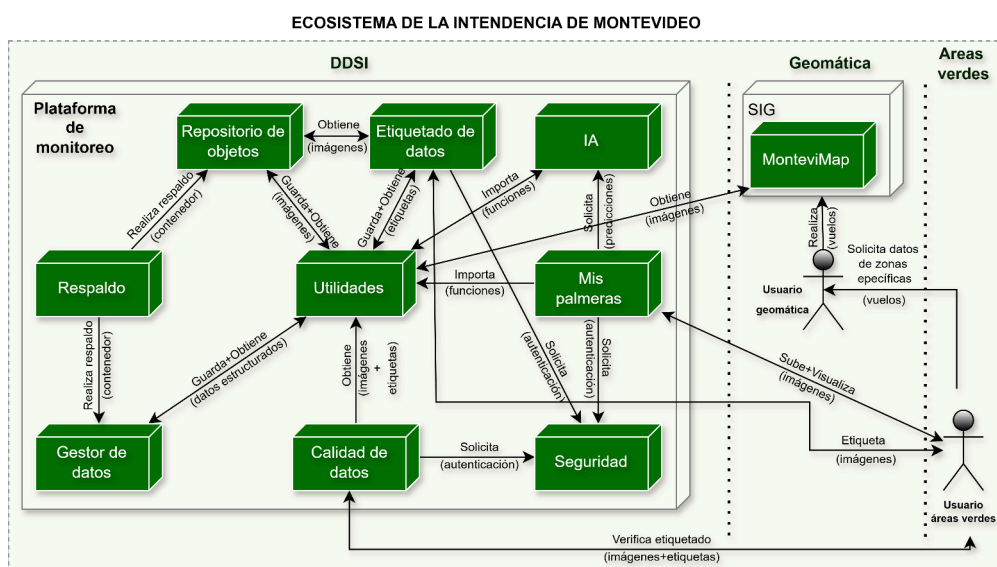


FIGURA 3.1. Diagrama de arquitectura de la plataforma y su ubicación en el ecosistema de la IM.

El enfoque se basó en un sistema modular, que favorece la encapsulación de funcionalidades específicas dentro de componentes con una única responsabilidad

(SRP, sigla del inglés *Single Responsibility Principle* [48]). Esto facilita la mantenibilidad y escalabilidad del sistema, lo que permite que cada módulo pueda ser desarrollado, probado, intercambiado y desplegado de manera independiente. Cada módulo desempeña un papel específico en varios de los procesos transversales de la ingeniería de software, como lo es la gestión de datos, la seguridad, el respaldo de información y el despliegue de aplicaciones.

La estructura modular de la plataforma permite que se puedan agregar otros módulos en el futuro, como por ejemplo, un módulo de información georreferenciada, que centralice las referencias de las palmeras y permita visualizaciones internamente utilizando herramientas como QGIS [49] o un módulo reportes que gestione todos los resultados de los entrenamientos de los modelos, con herramientas como MIFlow [50]. Esto posibilita que la plataforma pueda evolucionar y adaptarse a nuevas necesidades y tecnologías, lo que asegura su relevancia a largo plazo.

Uno de los principales objetivos al definir la arquitectura fue que los módulos representen responsabilidades que ya están resueltas por software existente en la infraestructura de la IM. Esto permite que, en un futuro, estos módulos puedan ser reemplazados por el software ya utilizado en la IM o incorporados como servicios adicionales aquellos que no existen, como el módulo de AI.

La comunicación se basa en protocolos ligeros como REST [35], que generan un bajo acoplamiento y una alta cohesión entre módulos, aunque también se utilizan otros protocolos como LDAP para la gestión de usuarios y permisos, así como OIDC para la autenticación y autorización de la aplicación web (Mis palmeras).

El módulo *Seguridad* permite la administración de usuarios y permisos. Esto garantiza que solo los usuarios autorizados tengan acceso a la información sensible. Se basa en el protocolo LDAP, que favorece una gestión centralizada de autenticación y autorización. Adicionalmente, puede ser sustituido por el software utilizado en la IM (WSO2 [51]) o directamente interactuar con este (WSO2 puede comunicarse mediante el protocolo LDAP).

El módulo *Respaldo de datos* cerciora que los datos estén siempre disponibles y que se puedan recuperar en caso de pérdida. En este caso es extremadamente importante este módulo, dado que el costo de obtener los datos etiquetados es muy alto, por lo que es información muy valiosa y esto aumenta el riesgo. El objetivo principal pasa por gestionar los procesos de respaldo y recuperación de datos. Al ser un módulo independiente, puede ser sustituido por otro software de respaldo, como Restic [52] o Bacula [53].

El módulo *Gestor de datos* es el encargado de almacenar los metadatos de las imágenes. Está conformado por una base de datos orientada a documentos, que permite almacenar varios tipos de datos (explicado en la sección 2.2.4). Esta forma de almacenamiento facilita la gestión de grandes volúmenes de información. Asimismo, esta estructura soluciona el desafío guardar las etiquetas generadas por los usuarios en el proceso de etiquetado de datos.

El módulo *Etiquetado de datos* permite ejecutar el proceso de etiquetado de imágenes y se comunica tanto con el módulo de seguridad para el manejo de diferentes niveles de información. Su acoplamiento es muy bajo, con el objetivo de que pueda ser sustituido por otro software de etiquetado, dado la gran variedad de herramientas disponibles en el mercado. Se comunica con el módulo de

repositorio de objetos para obtener las imágenes a etiquetar y con el módulo de seguridad para gestionar los usuarios y permisos. Asimismo, brinda una interfaz gráfica para que el usuario pueda realizar el etiquetado de manera sencilla.

El módulo `Calidad de datos` se ocupa de realizar un análisis cualitativo de los datos, lo que mejora la eficiencia de los modelos al utilizar datos de alta calidad. También permite que un equipo de control de calidad pueda analizar y generar informes específicos sobre los datos, lo que habilita una traza y evolución de las imágenes, esencial en proyectos de CV donde la variable temporal afecta el resultado de las predicciones (como sucede en este trabajo). Por último, se comunica con el usuario mediante una interfaz gráfica que permite una correcta gestión de la calidad de los datos.

El módulo `Repositorio de objetos` es el responsable del almacenamiento y gestión de las imágenes, en este caso las generadas por drones. Administra grandes volúmenes de datos y garantiza la disponibilidad continua de esta información mediante APIs. Para esta tarea, usualmente se utilizan sistemas de almacenamiento de objetos como Amazon S3 [54] o algún otro proveedor de nube. La comunicación con otros módulos se realiza mediante APIs REST, lo que permite una integración sencilla y eficiente.

El módulo `Mis palmeras` representa las aplicaciones webs principales del sistema. Es la cara visible de éste y permite a los usuarios interactuar con la información de manera sencilla. La aplicación web primordial tiene el objetivo de mostrar el estado de la plaga en base a las imágenes aéreas y los resultados de los modelos de detección. Se comunica con el módulo de seguridad para la autenticación y autorización de los usuarios, utiliza componentes del módulo de utilidades (como las transformaciones entre varios formatos) y solicita predicciones al módulo de AI. Está orientado a mostrar información georreferenciada y actualmente depende fuertemente del módulo AI, sin embargo, en un futuro se podría mejorar esta interacción mediante una API REST, lo que permitiría una mayor flexibilidad y escalabilidad del sistema.

El módulo `Utilidades` centraliza aplicaciones utilizadas en varias tareas, como la actualización, migración y generación de datos que usualmente son insumos para los procesos de entrenamiento. Este módulo realiza tareas específicas que no están directamente relacionadas con el proceso de detección, pero que son necesarias para el correcto funcionamiento del sistema. Un ejemplo de esto es la aplicación que efectúa *web scrapping* del sistema de información geográfica de la IM, para descargar las imágenes de los vuelos allí publicadas y guardarlas en el repositorio de imágenes.

Finalmente, el módulo `AI` se encarga de cumplir con los procesos asociados al entrenamiento y despliegue de los modelos. Tiene el objetivo de manejar sus ciclos de vida, desde el desarrollo hasta su despliegue en producción e incluye la ejecución de experimentos. Actualmente no expone una API para solicitar predicciones, lo que lo hace menos flexible (hay que estar gestionando los artefactos de un módulo a otro).

Actualmente el sistema está alojado en una infraestructura autogestionada y un resumen de las interfaces de comunicación con el exterior de la plataforma se puede observar en la figura 3.2, en donde se puede notar que existen cuatro interfaces de comunicación externa: tres interfaces webs y una API REST para realizar el *web scrapping*.

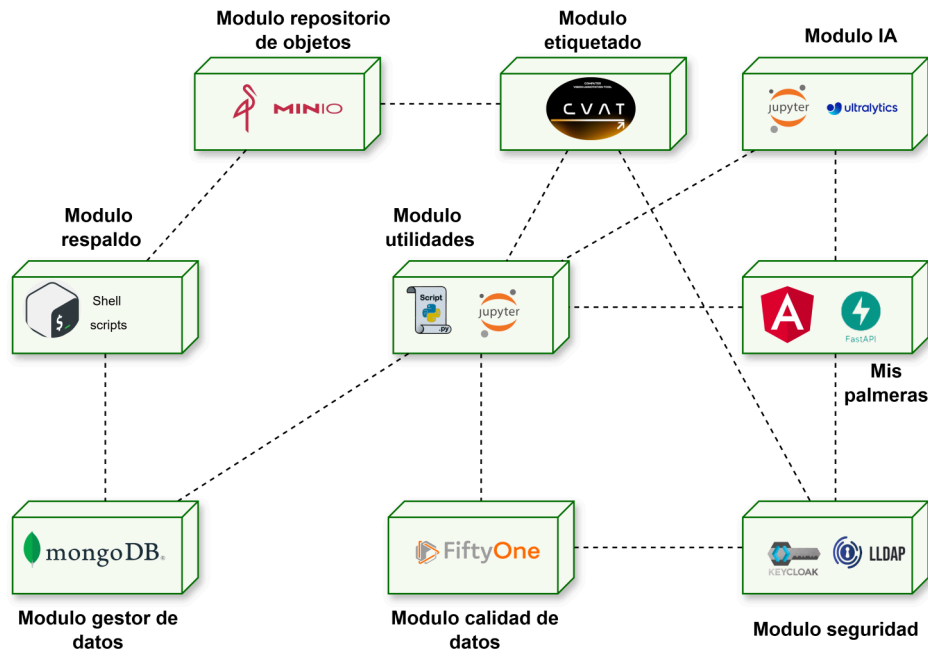


FIGURA 3.3. Diagrama de bloques del sistema con sus tecnologías utilizadas.

Una descripción de las tecnologías utilizadas en cada módulo se puede observar en la tabla 3.1.

El despliegue de la infraestructura de producción pasó por tres etapas de adaptación:

- **Despliegue inicial en la nube pública:** se utilizó la nube pública de Scaleway para el despliegue inicial, aprovechando su capa gratuita para almacenamiento de objetos (*object storage*), que permite almacenar hasta 80 GB de datos. Esto permitió validar la arquitectura y las interacciones entre los módulos en un entorno realista. Surgieron varios desafíos, como la correcta selección de las instancias de cómputo adecuadas para CVAT, la configuración de la red para permitir el acceso externo y la gestión de costos asociados al uso de recursos en la nube. Además, para el despliegue de CVAT se gestionó certificados SSL utilizando Let's Encrypt [57], lo que agregó otra capa de complejidad al proceso de configuración inicial. Sin embargo, a medida que el proyecto avanzaba, los costos aumentaron significativamente, por lo que se decidió migrar a una infraestructura semigestionada.
- **Despliegue semigestionada:** se optó por una infraestructura semigestionada con un servidor físico local. Esto permitió reducir costos y tener un mayor control sobre los recursos. Se utilizó Ngrok [58] (un servicio de *tunneling*) para exponer los servicios a Internet de manera segura, lo que facilitó el acceso remoto a la plataforma. Esta etapa implicó desafíos relacionados con la configuración de la red interna, la gestión de certificados SSL y la optimización del rendimiento de los servicios desplegados. Si bien se disminuyeron los costos en comparación con la nube pública, no se llegó a un costo aceptable, dado que Ngrok tiene un tráfico de salida limitado y como se trabajó con imágenes de alta resolución, este límite se alcanzaba rápidamente. Por

TABLA 3.1. Tecnologías utilizadas por módulo.

Módulo	Herramienta	Tecnología/Descripción
Seguridad	LLDAP, Self Service Password [55], KeepAlive, Keycloak, Traefik	LDAP, PHP, Bash, OIDC, OAuth2, Go
Respaldo	Backup-Restore, Scripts personalizados	Bash
Gestor de datos	MongoDB	Base de datos NoSQL
Etiquetado de datos	CVAT	React, Django, PostgreSQL
Calidad de datos	FiftyOne	React, Python (ASGI), MongoDB
Repositorio de objetos	MinIO	Go
Mis palmeras	Aplicación web	FastAPI, Angular
Utilidades	Web scrapping, anotaciones	Python, BeautifulSoup, CVAT SDK
AI	YoloV11, anotadores	Python, PyTorch, Ultralytics
Infraestructura	Docker Compose, Vagrant	Docker, VirtualBox [56]

lo tanto, se decidió migrar a una infraestructura autogestionada como solución final.

- Despliegue final (autogestionado): existieron muchos desafíos en esta etapa, desde la configuración de servicios *no-ip* para gestionar un DNS dinámico, hasta la correcta configuración de los certificados SSL. Para esto, se contrató un dominio en Cloudflare [59] (picudo-rojo.org), se configuró Traefik [60] como proxy reverso y se realizaron diversas configuraciones para exponer de forma segura una máquina virtual por medio de Vagrant. Esta etapa finalizó con el despliegue exitoso de la plataforma en un entorno autogestionado, lo que aseguró su disponibilidad y rendimiento óptimo.

El ambiente de desarrollo se puede reproducir en cualquier máquina que tenga instalado Vagrant y VirtualBox. La máquina anfitriona puede ser cualquier sistema operativo, ya sea Windows, macOS o Linux. Esto permite que cualquier desarrollador pueda tener el mismo ambiente de desarrollo en su máquina local, lo que facilita la colaboración y el trabajo en equipo, simplemente clonando el repositorio [61] mediante Git e importando el proyecto con un IDE [62]. En este caso, se utilizó Visual Studio Code.

Una vez importado el proyecto, la estructura de directorios que emula cada módulo del proyecto se puede apreciar en el código 3.1.

```

1 desarrollo/
2 |--- modulo-aplicaciones-web/
3     |--- entrypoint/
4     |--- mis-palmeras/
5     |--- landing-page/
6 |--- modulo-calidad-datos/
7     |--- fiftyone/
8 |--- modulo-etiquetado-datos/
9     |--- cvat/
10 |--- modulo-gestor-datos/
11     |--- mongo/
12 |--- modulo-ia/
13     |--- modulo_ia/
14     |--- ...
15 |--- modulo-utilidades/
16     |--- modulo_utils/
17     |--- ...
18 |--- modulo-repositorio-objetos/
19     |--- minio/
20 |--- modulo-respaldo/
21     |--- scripts/
22 |--- modulo-seguridad/
23     |--- keepalive/
24     |--- keycloak/
25     |--- ssp/
26     |--- ldap/
27 |--- vagrant-scripts/
28 |--- readme.md
29 |--- Vagrantfile

```

CÓDIGO 3.1. Estructura de directorios utilizada.

Los módulos AI y utilidades, se estructuran utilizando Cookiecutter Data Science [63], que es una plantilla para proyectos de ciencia de datos. Esta estructura facilita la organización del código, datos y documentación, y promueve buenas prácticas en el desarrollo de proyectos de ciencia de datos. Estos módulos también cuentan con UV Python como gestor de dependencias y entornos virtuales, lo que permite aislar las dependencias de cada proyecto y evitar conflictos entre ellas. Finalmente, también posibilita empaquetar y distribuir las funcionalidades desarrolladas como librerías reutilizables.

En ingeniería de software es importante asegurar la reproducibilidad de los ambientes de trabajo. Una de las herramientas utilizadas con este objetivo fue Docker, que permite crear contenedores que encapsulan una aplicación y sus dependencias. Esto asegura un entorno de ejecución uniforme y reproducible en diferentes sistemas. Para la orquestación de múltiples contenedores se utilizó Docker Compose, que brinda una infraestructura en entornos complejos en donde interactúan varias aplicaciones.

Cada directorio tiene su propio archivo de Docker Compose (`compose.yml`) que habilita el despliegue de las herramientas del módulo. Esto, además de las ventajas mencionadas anteriormente en la sección 3.1, también permite que cada componente pueda ser desarrollado y probado de manera independiente (con sus debidos *stubs* [64] o *mocks* [65]), lo que reduce el *lead time* [66]. Estos archivos de configuración pueden ser adaptados a distintas situaciones. Por ejemplo, el archivo `compose.yml` de MinIO fue adaptado para que, si no existe el *bucket* llamado *picudo-rojo-bucket*, se cree automáticamente.

Asimismo, cada módulo tiene sus variables de entorno definidos como archivos `.env` su directorio raíz. Esto permite separar la configuración de los distintos ambientes para cada módulo. En este sentido, facilita la transición entre el ambiente de desarrollo y el ambiente de producción.

Aunque Docker permite crear contenedores que emulan el ambiente de producción, esto no es suficiente para asegurar la reproducibilidad de este dominio en el entorno de desarrollo. Esto se debe a que Docker se ejecuta sobre una plataforma, que puede ser diferente en cada máquina. Por lo tanto, es necesario utilizar una herramienta que permita crear y gestionar la infraestructura de soporte de manera uniforme y reproducible. Para esto se utilizó Vagrant.

Vagrant concede la habilidad de crear una o varias máquinas virtuales (VM, del inglés *virtual machine*) que emulan una plataforma objetivo, en este caso, el ambiente de producción (Ubuntu 24). Se maneja con un archivo de configuración llamado Vagrantfile que brinda la posibilidad de realizar la definición de la configuración de la máquina virtual como *infrastructure as a code*. Esto implica definir parámetros como la cantidad de memoria, el sistema operativo o las aplicaciones a instalar. Se puede utilizar una imagen de un sistema operativo ya configurado o crear una desde cero. En este caso se utilizó una imagen de Ubuntu 24.04 LTS [67], del repositorio oficial de Vagrant [68], que incluye la mayoría de las herramientas necesarias para el desarrollo y despliegue del sistema. Esta imagen se puede utilizar en cualquier máquina que tenga instalado Vagrant y VirtualBox [56] (también se puede utilizar otro software de virtualización, como lo es Hyper-V [69] o VMWare [70]).

Asimismo, Vagrant también permite aprovisionar *scripts* de configuración a las máquinas virtuales. Estos pueden ejecutarse al iniciar estas máquinas, lo que permite instalar y configurar las aplicaciones necesarias para poner el funcionamiento el sistema. En este caso, se utilizaron cuatro scripts de aprovisionamiento que pueden apreciarse en el código 3.2.

```
1 vagrant-scripts /
2 |--- 01-setup-network.sh
3 |--- 02-install-dependencies.sh
4 |--- 03-configure-firewall.sh
5 |--- 04-start-dev-services.sh
```

CÓDIGO 3.2. Scripts de vagrant utilizados.

En donde el último se encarga de iniciar los servicios dentro de cada módulo (usualmente se ejecuta mediante un script de shell el comando `docker compose up`).

Una vez que se ejecuta el comando `vagrant up`, se crea una máquina virtual (si ya no fue creada), y se ejecutan los scripts de aprovisionamiento, lo que resulta en el ambiente de desarrollo listo para utilizarse. La ejecución del ambiente de producción es exactamente igual, pero con distintas configuraciones de archivos. Finalmente, se configuró el router para redirigir el tráfico de los puertos 80 y 443 de la máquina virtual, lo que permite acceder a la plataforma desde Internet.

Entre los servicios que se inician se encuentran:

- MongoDB: implementa la base de datos que permite almacenar metadatos de las imágenes ortorectificadas.
- MinIO: almacena las imágenes generadas por los drones.

- CVAT: se utiliza para etiquetar las imágenes almacenadas en el repositorio de objetos.
- FiftyOne: herramienta de análisis, organización, visualización y generación de informes sobre los datos.
- LLDAP: herramienta de gestión de usuarios y permisos, utilizada para administrar la seguridad del sistema.
- LDAP *Self Service Password*: autoservicio de contraseñas (permite que los usuarios gestionen sus credenciales)
- *Landing Page*: conjunto de herramientas basadas en tecnologías de desarrollo web (HTML, CSS y Javascript) que brindan información sobre el trabajo. Es la cara visible del sistema desde Internet.
- Mis Palmeras: aplicación web principal del sistema, que permite a los usuarios interactuar con la información de manera sencilla.
- Keycloak: sistema de gestión de identidad y acceso, que permite la autenticación y autorización de los usuarios.
- *Entrypoint*: proxy reverso que expone el ambiente de desarrollo a Internet o intranet para su fácil acceso.

Una vez iniciados los servicios, se puede acceder a la plataforma desde Internet mediante el dominio <https://picudo-rojo.org>, lo que muestra la página de inicio (*landing page*) del sistema, como se observa en la figura 3.4.



FIGURA 3.4. Captura de pantalla de la página de inicio (*landing page*) del sistema, desde donde se puede acceder a la aplicación web principal.

Desde allí, se puede acceder a la aplicación web principal (Mis palmeras) mediante el dominio <https://app.picudo-rojo.org>, al sistema de etiquetado de imágenes mediante el dominio <https://cvat.picudo-rojo.org> y la gestión de las contraseñas mediante el dominio <https://password.picudo-rojo.org>.

3.3. Obtención y gestión de las imágenes

Posterior al despliegue de la infraestructura, se procedió a la obtención y gestión de las imágenes ortorectificadas. Para ello, se desarrolló un proceso de *web scrapping* que automatiza la descarga de las imágenes desde el SIG.

El SIG de la IM es un visor web de mapas que está construido con un framework que utiliza MapServer [71], en donde se permite navegar por capas de información geográfica. En la figura 3.5 se puede observar una captura de pantalla, con la grilla de las imágenes de drones ortorectificadas de Montevideo.

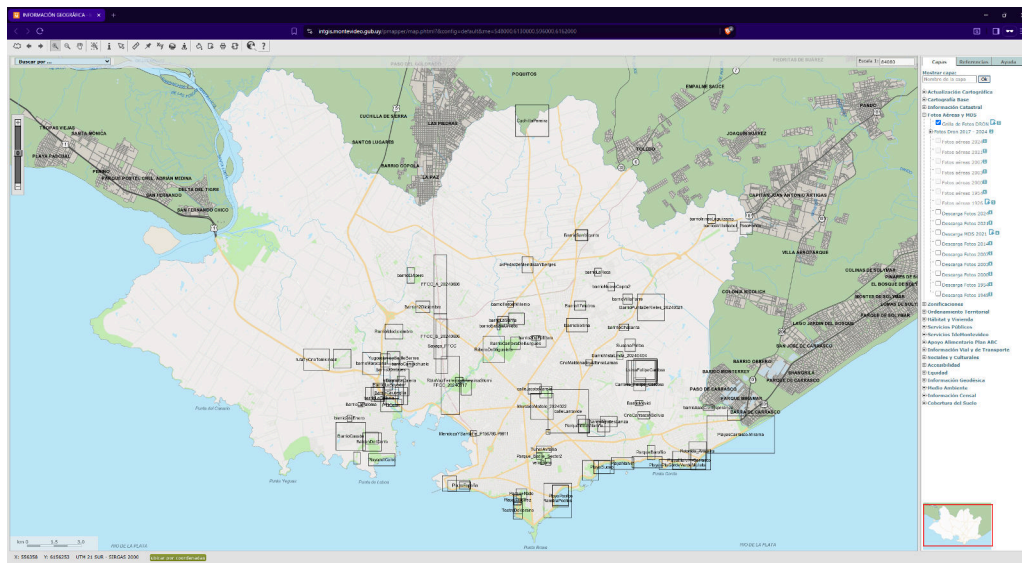


FIGURA 3.5. Captura de pantalla del SIG de la IM, que muestra la grilla de las imágenes ortorectificadas de drones en Montevideo.

Los vuelos fueron realizados con los drones DJI Phantom 4 RTK y DJI Mavic 2 Pro, ambos equipados con cámaras de alta resolución. Las imágenes capturadas por los drones fueron procesadas utilizando software especializado de fotogrametría, que incluyó algoritmos avanzados para la corrección geométrica y la ortorectificación. Este proceso aseguró que las imágenes fueran precisas y estuvieran alineadas correctamente con las coordenadas geográficas, lo que es esencial para su uso en análisis geoespaciales y aplicaciones de monitoreo ambiental.

Las imágenes ortorectificadas tienen una resolución espacial que varía entre 3 y 5 cm por píxel, dependiendo de la altitud del vuelo y las condiciones de captura, y están publicadas en formato vectorial (.jpg). En la obtención de las imágenes, se incluyeron ambas resoluciones.

El proceso de *web scrapping* se desarrolló utilizando Python y la librería BeautifulSoup [72], que permite extraer información de páginas web de manera sencilla. El script automatiza la navegación por el SIG, identifica las imágenes ortorectificadas disponibles de los drones y las descarga en el repositorio de objetos (MinIO). Este proceso se ejecuta a demanda, lo que permite mantener el repositorio actualizado con las últimas imágenes disponibles en el SIG. Un desafío importante consistió en identificar la estructura HTML del SIG y extraer correctamente las URLs de las imágenes, lo que requirió un análisis detallado del código fuente de

la página web, que derivó en la conclusión de que las imágenes estaban almacenadas en un servidor externo, que primero se generaban temporalmente y luego se permitía la descarga. Otro punto a destacar es que esta información estaba en un archivo Javascript que se generaba dinámicamente, lo que complicó la extracción de las URLs. Sin embargo, una vez superados estos desafíos, el proceso de *web scrapping* se implementó con éxito, lo que permitió la obtención eficiente de las imágenes ortorectificadas.

Un tema de gran importancia sobre las imágenes fue que el tamaño de estas era muy grande, lo que impidió que se cargaran tal cual en CVAT, dado que la herramienta tiene un límite de 65 000 píxeles en el lado más largo de la imagen. Por lo tanto, se decidió recortar las imágenes en fragmentos más pequeños, de 4096×4096 px, lo que permitió que se pudieran cargar sin problemas. Este proceso de recorte se automatizó con el objetivo de facilitar la descarga de todas las imágenes. También se organizó el repositorio de objetos de manera estructurada y se utilizó un esquema de nombres que facilitó la identificación y gestión de las imágenes. Un ejemplo de los recortes realizados se puede observar en la figura 3.6.



FIGURA 3.6. Ejemplos de recortes de las imágenes ortorectificadas utilizadas.

La organización de las imágenes en MinIO se realizó utilizando una nomenclatura altamente acoplada a CVAT, en donde cada carpeta representa un *task* (o tarea

de etiquetado) de CVAT. Esto permite que se puedan agregar *tasks* al proyecto sin problemas. Dentro de cada *task*, las imágenes se nombraron utilizando el nombre común (obtenido desde la página del SIG) y su número de parche (el recorte anteriormente mencionado). La estructura puede observarse en el código 3.3.

```

1 picudo-rojo-bucket
2 |--- imagenes_metadatos
3 |--- imagenes
4 |   |--- task-training-1
5 |       |--- AcostayLara_20240927_dji_rtk_5cm.jpg
6 |   |--- task-training-2
7 |   |--- ...
8 |--- parches
9 |   |--- task-training-1
10 |       |--- AcostayLara_20240927_dji_rtk_5cm
11 |           |--- AcostayLara_20240927_dji_rtk_5cm_patch_3.jpg
12 |           |--- ...
13 |       |--- ...

```

CÓDIGO 3.3. Estructura de las imágenes en MinIO.

La división en estos grupos también permite separar cuales imágenes son de entrenamiento y cuales son de *testing*. Simplemente al agregar un nuevo grupo llamado *task-testing*, se puede separar las imágenes de *testing* de las de entrenamiento. En este trabajo no se utilizó dicha separación, dado que la división se realizó después por la poca cantidad de datos de las clases minoritarias.

A la vez que se guardaban las imágenes en MinIO, se almacenaban los metadatos en MongoDB. Estos metadatos incluyen información relevante sobre cada imagen, como su nombre, resolución, fecha de captura y coordenadas geográficas. Esta información es esencial para la gestión y análisis de las imágenes, ya que permite realizar búsquedas y filtrados basados en diferentes criterios. La estructura de los documentos en MongoDB se diseñó para facilitar la consulta y actualización de los metadatos, asegurando una gestión eficiente de la información. Un aspecto importante a destacar es el nodo *jgw_data* dentro de la estructura. Este nodo contiene la información necesaria para georreferenciar la imagen, lo que permite que se pueda ubicar en un sistema de coordenadas geográficas. Esta información está adjunta con la imagen cuando se descarga desde el SIG.

Una vez que se completó la descarga de las imágenes, se procedió a cargarlas en CVAT para su posterior etiquetado. Este proceso se realizó de manera manual, se utilizó la interfaz gráfica de CVAT para seleccionar y cargar las imágenes desde MinIO. Se creó un proyecto en CVAT llamado *picudo-rojo-3cm-5cm* y se agregaron las imágenes correspondientes a este trabajo. Este proceso fue sencillo y eficiente, gracias a la integración entre CVAT y MinIO, lo que permitió una gestión fluida de las imágenes.

3.4. Proceso de etiquetado de datos

El proceso de etiquetado de datos fue una etapa crucial. Inicialmente, se intentó utilizar la información georreferenciada de las palmeras proporcionada por la IM. Sin embargo, se descubrió que los recortes de las imágenes no eran exactos, lo que dificultaba la identificación precisa de las palmeras en las imágenes ortorectificadas debido a la gran variedad de tamaños, formas y colores que presentan las palmeras en las imágenes aéreas. La principal tarea en este caso era

la de ajustar y verificar las coordenadas de las palmeras en las imágenes, lo que resultó ser un proceso laborioso y propenso a errores. Por lo tanto, se optó por un enfoque manual para este etiquetado. Para esto, se contaba con el apoyo de un equipo de ingenieros del sector de arbolado de la IM. Específicamente, se contó con la colaboración de cuatro ingenieros agrónomos, quienes tenían experiencia en el manejo y cuidado de palmeras, lo que resultó fundamental para la correcta identificación y etiquetado de éstas.

Se definieron los usuarios y roles, se cargaron los *tasks* de CVAT y se dejó en libertad que cada usuario trabajara a su ritmo, asignándose un *job* según sus necesidades. Se realizaron varias jornadas de etiquetado, en donde se evacuaron dudas y se definieron lineamientos para el etiquetado. Se establecieron cuatro clases de etiquetas: palmera-sana, palmera-infectada, palmera-muerta y palmera-exterminada. Un ejemplo de cada clase puede observarse en la figura 3.7.

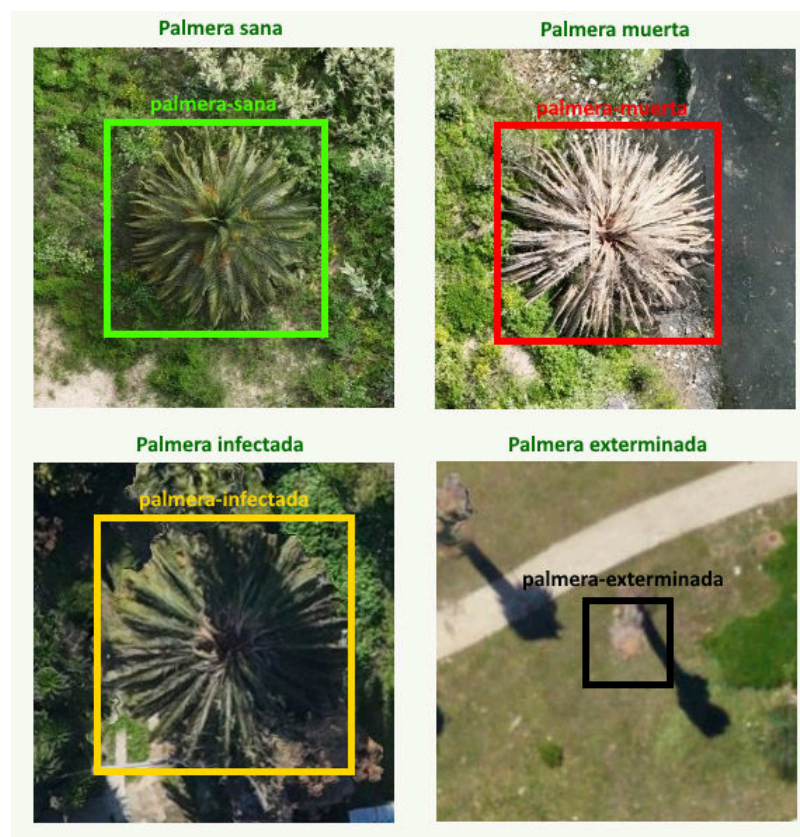


FIGURA 3.7. Ejemplos de las clases definidas para el etiquetado de palmeras.

CVAT tiene la ventaja de permitir definir atributos para cada etiqueta. Se utilizó esta funcionalidad para definir los atributos *especie* y *dudoso*. El primero se definió como una lista de todas las especies de palmeras identificadas en Montevideo, el segundo simplemente para marcar si la palmera estaba borrosa o no, en casos donde hubiese alguna duda.

La comunicación fue un aspecto fundamental durante el proceso de etiquetado. Se utilizó una combinación de correos electrónicos, manuales, reuniones presenciales y un grupo de WhatsApp para mantener una comunicación fluida entre

los miembros del equipo. Esto permitió generar un consenso sobre las mejores prácticas para el etiquetado y resolver dudas de manera rápida y eficiente, lo que aumentó la calidad de los datos etiquetados.

Aún con todo el apoyo brindado, la tarea de etiquetado resultó ser más desafiante de lo esperado. La complejidad de las imágenes, la variabilidad en la apariencia de las palmeras y la necesidad de un etiquetado preciso hicieron que el proceso fuera laborioso y requiriera una atención meticulosa a los detalles. Esto llevó a que el progreso del etiquetado fuera más lento de lo anticipado, lo que impactó en los plazos del proyecto. A pesar de estos desafíos, se logró avanzar significativamente en el etiquetado de las imágenes, lo que sentó las bases para el desarrollo de los modelos de detección.

Se etiquetaron un total de 2 204 imágenes, en donde se identificaron cerca de 9 000 palmeras. Un resumen detallado se puede observar en la tabla 3.2.

TABLA 3.2. Resumen del proceso de etiquetado de imágenes.

Concepto	Cantidad
Ortomosaicos totales	122
Parches totales (incluidos blancos)	4 174
Parches totales (sin blancos)	2 204
Parches etiquetados	2 204
Promedio de parches por ortomosaico	18,07
Parches con palmeras	1 288
Total de palmeras etiquetadas	9 044
Palmeras sanas	8 403
Palmeras infectadas	170
Palmeras muertas	280
Palmeras exterminadas	191

La cantidad de etiquetas de la clase *palmera-sana* con respecto a las demás, se debe a que la mayoría de las imágenes fueron tomadas antes de la llegada del picudo rojo. En este punto se identificó un grave problema, en donde no se iba a lograr obtener una cantidad suficiente de detecciones con palmeras infectadas, muertas o exterminadas, lo que implicó utilizar otros enfoques para el entrenamiento de los modelos que se explican en siguiente capítulo.

Es muy difícil obtener una métrica certera de cuantas etiquetas por imagen se realizaron, dado que no todas las imágenes tenían palmeras y algunas tenían muchas más que otras. Tampoco se pudo determinar el tiempo exacto de etiquetado total (pero se estima que rondó cerca de 40 hs). Se pudo evidenciar que hay imágenes que llegaban a contener hasta 50 palmeras, lo que da una idea de la complejidad del trabajo realizado. Estos datos se pudieron obtener gracias a la herramienta FiftyOne, que permite analizar y visualizar los datos de manera sencilla. Un resumen de algunas jornadas del proceso de etiquetado se puede observar en la tabla 3.3.

TABLA 3.3. Resumen de algunas jornadas del proceso de etiquetado.

Jornada	Horas	Imágenes	Palmeras
1	2,5	100	650
2	2,1	75	465
3	3,0	90	720
4	2,8	140	480
Promedio	2,3	95	524

Un ejemplo de una imagen etiquetada en CVAT se puede observar en la figura 3.8, en donde se puede ver una palmera sana (verde), una infectada (amarillo) y dos muertas (rojo).

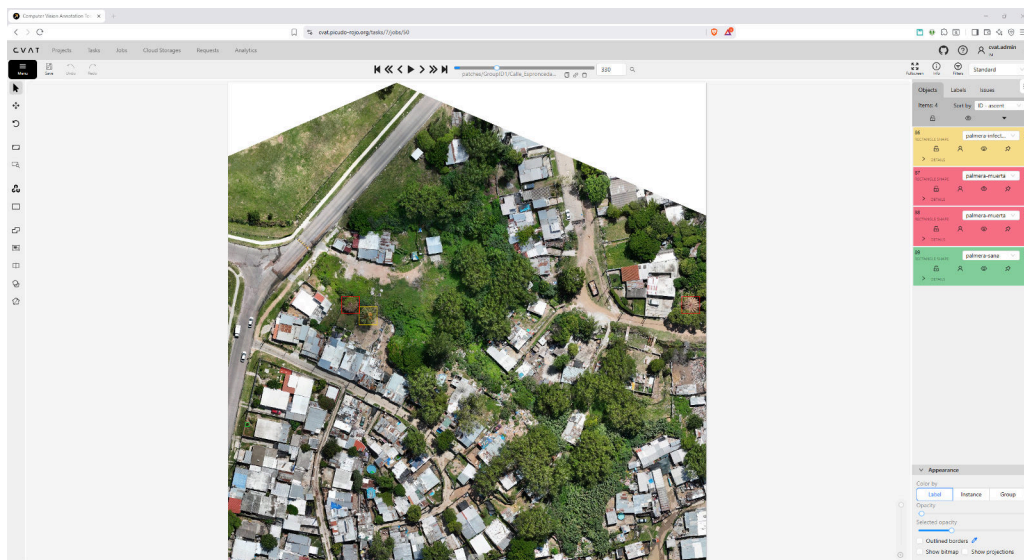


FIGURA 3.8. Ejemplo de una imagen etiquetada en CVAT.

3.5. Preparación del dataset y estructura del entrenamiento

La estructura de entrenamiento se basó en la plantilla de Cookiecutter Data Science, que permitió organizar los datos y scripts de manera eficiente y reproducible. En el código 3.4 se puede observar la plantilla adaptada a la estructura de directorios para el módulo AI.

```

1 modulo-AI
2 |--- data
3 |   |--- iterim
4 |   |--- processed
5 |       |--- yolo_palm_detection_dataset_v1.0
6 |       |--- yolo_rpw_detection_dataset_v1.0
7 |   |--- raw
8 |       |--- coco_palm_dataset_v1.0
9 |   |--- temp
10 |--- models
11 |       |--- coco_palm_detection_yolo11n_640.pt

```

```

12 |   |-- coco_palm_detection_yolo11x_640.pt
13 |   |-- coco_rpw_detection_yolo11n_640.pt
14 |   |-- coco_rpw_detection_yolo11x_640.pt
15 | |-- modulo_ia
16 |     |-- modeling
17 |     |-- utils
18 |     |-- config.py
19 |     |-- dataset.py
20 |     |-- features.py
21 |     |-- main.py
22 | notebooks
23 | |-- deteccion_palmeras
24 |     |-- yolov11
25 |         |-- 3.01-bm-deteccion-palmeras-yolov11.ipynb
26 | |-- deteccion_picudo_rojo
27 |     |-- yolov11
28 |         |-- 3.02-bm-deteccion-rpw-yolov11.ipynb
29 | uv.lock
30 | pyproject.toml

```

CÓDIGO 3.4. Estructura de directorios utilizada.

El dataset se organizó en dos directorios:

- `raw dataset`: contiene las imágenes originales sin procesar, junto con sus anotaciones en formato COCO. Estas imágenes fueron descargadas desde el almacenamiento en la nube (MinIO) y sus metadatos desde el gestor de datos (MongoDB). Representan el conjunto completo de datos disponibles para el entrenamiento. Se descargaron únicamente aquellos parches que contenían anotaciones (se puede obtener más detalles en la tabla 3.2 de este capítulo).
- `processed dataset`: contiene las imágenes procesadas y preparadas para el entrenamiento del modelo que son consumidas directamente por el modelo para el entrenamiento, validación y *test*.

Esta organización recomendada por Cookiecutter Data Science permite mantener un flujo de trabajo claro y estructurado, en donde el `raw dataset` permanece inalterado, mientras que el `processed dataset` puede ser modificado y ajustado según las necesidades del entrenamiento. El directorio `interim` se utiliza para almacenar datos intermedios generados durante el procesamiento del dataset final, y el directorio `temp` se emplea para datos temporales que no necesitan ser versionados. En este caso, no se guardó los datos intermedios ni temporales, por lo que ambos directorios permanecen vacíos. La descarga del dataset se realizó utilizando scripts de Python que interactúan con la API de MinIO para obtener las imágenes, sus correspondientes anotaciones de la base de datos y finalmente la conversión al formato COCO. La clase `dataset.py` centraliza todas estas funcionalidades.

El procesamiento del dataset final incluyó una versión del `raw dataset` que pasó por varias etapas, como la conversión de las anotaciones al formato YOLO, el recorte de las imágenes en subimágenes más pequeñas para mejorar la detección utilizando SAHI [73] (método en donde se divide una imagen en varias imágenes más pequeñas), el balanceo del dataset para abordar la desproporción entre las clases y la creación de los *splits* de entrenamiento, validación y *test*. Se utilizó la herramienta FiftyOne para la visualización y validación del dataset, así como para obtener las métricas de calidad.

Dentro de estas etapas de procesamiento se destaca el recorte de las imágenes para la utilización de SAHI, que demoró usualmente cerca de 20 minutos en procesarse; el balanceo del dataset, en donde se decidió aplicar *undersampling* [74], que reduce la cantidad de detecciones de las clases mayoritarias (palmeras sanas) para igualar la cantidad de detecciones de las clases minoritarias (palmeras infectadas, exterminadas y muertas), que demoró también cerca de 20 minutos en procesarse. La aplicación de *undersampling* se debió a la poca cantidad de datos disponibles para las clases minoritarias, lo que dificultaba la aplicación de técnicas de *oversampling* sin incurrir en un sobreajuste significativo. Un resumen de las operaciones aplicadas al dataset y sus directorios de salida se presenta en la tabla 3.4.

TABLA 3.4. Resumen de las operaciones aplicadas al dataset durante el procesamiento.

Etapa	Directorio de salida
Dataset original (COCO)	raw
Conversión a formato YOLO	interim
Recortes de 640×640	interim
Balanceo de clases	interim
Splits finales (80 %/10 %/10 %)	processed

El recorte en imágenes más pequeñas para el entrenamiento en este caso fue muy importante, dado que las palmeras ocupaban una pequeña porción de la imagen original. En CV las palmeras se clasificaban como en *tiny objects* (con respecto a los ortomosaicos). Estos son objetos que ocupan menos del 0,1 % del área total de la imagen [75]. En este caso ocupaban aproximadamente el 0,005 % del área total de la imagen. Si bien era necesario el recorte de las imágenes por complejidades técnicas (de memoria de la GPU), también era necesario para mejorar la detección de las palmeras. Con este objetivo, se ideó un proceso de recorte de las imágenes en subimágenes de 640×640 píxeles, con un solapamiento de 250 píxeles entre cada recorte. Este tamaño se eligió dado que era el tamaño con el que el modelo YoloV11 había sido preentrenado (dataset de COCO [76]). En estos recortes las palmeras representaban cerca de un 30 % del área total de la subimagen, por lo que pasaban a ser simplemente objetos normales.

Un ejemplo de este proceso de recorte se puede observar en la figura 3.9, en donde un parche podría ser recortado en 40 subimágenes de 640×640 píxeles cada una, muchas que no contienen palmeras.

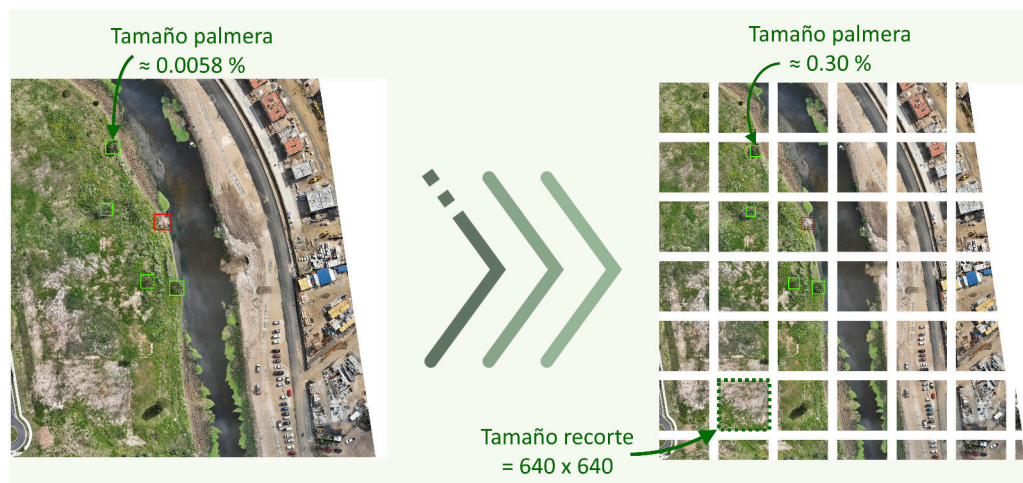


FIGURA 3.9. Ejemplo del proceso de recorte de imágenes.

Este proceso involucró desafíos como casos en donde las palmeras quedaban divididas entre dos recortes o donde el recorte en su mayoría era blanco. Para el primer caso, se adaptaron las anotaciones en base a un umbral (0,1 %). Si este umbral era superado, la palmera se incluía en el recorte correspondiente; de lo contrario, se descartaba. En el segundo caso, se utilizó una función del módulo utilidades (la misma utilizada para el recorte de parches para CVAT). Esta función permite identificar imágenes con un porcentaje alto de píxeles blancos y descartarlas automáticamente. Esto permitió optimizar el dataset final, eliminando imágenes que no aportaban valor al entrenamiento.

3.6. Desarrollo de la aplicación web

En paralelo al proceso de etiquetado, se desarrolló la aplicación web principal del sistema, llamada Mis Palmeras. Esta aplicación tiene como objetivo principal mostrar los resultados de la detección del picudo rojo en las palmeras, utilizando las imágenes aéreas y los modelos de detección desarrollados.

El principal caso de uso comienza con la interacción por fuera de la plataforma de los usuarios del sector de arbolado y del servicio de geomática de la IM. Inicialmente, los primeros identifican zonas en donde se sospecha la presencia del picudo rojo, basándose en reportes ciudadanos o áreas en donde se expande la plaga. Esta información se comunica al servicio de geomática (mediante una petición formal), que se encarga de planificar vuelos con drones y preprocesar las imágenes aéreas de estas áreas. Una vez que las imágenes son capturadas y procesadas, los usuarios de arbolado acceden a la aplicación para cargarlas y solicitar el análisis. La aplicación procesa las imágenes y brinda los resultados de la detección. Este proceso permite mejorar la toma de decisiones, ya que según la cantidad de palmeras en estados delicados, se puede decidir si es necesario un monitoreo más exhaustivo o la implementación de medidas de control específicas.

La figura 3.10 ilustra este flujo de trabajo. Se puede observar que en el paso 5 los usuarios interactúan con la aplicación web para cargar las imágenes y recibir los resultados del análisis.

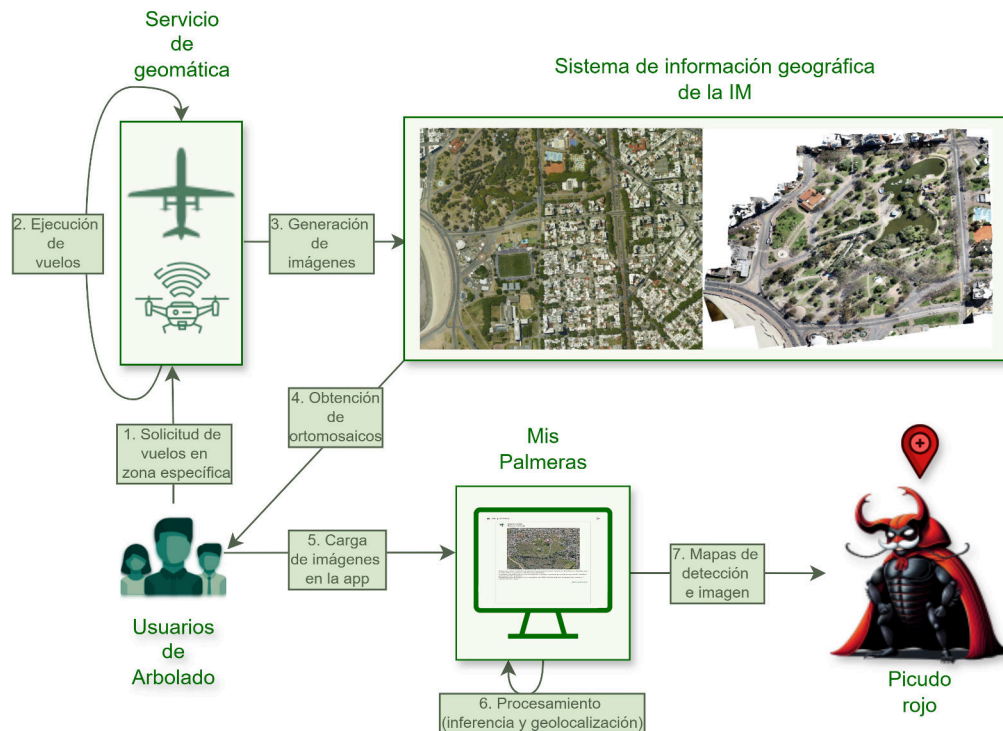


FIGURA 3.10. Diagrama del caso de uso principal de la plataforma.

El valor final al usuario es un mapa con las palmeras detectadas y clasificadas por su estado de salud. Esta información se puede descargar en un KML para importar en otras herramientas utilizadas por varios equipos que gestionan la expansión de la plaga. También se puede descargar la imagen para verificar la visualización de las detecciones.

El desarrollo de la aplicación web se realizó con Angular para el frontend y FastAPI para el backend. Se utilizó Keycloak como capa de autenticación y autorización, lo que permite gestionar los usuarios y sus permisos de manera segura. La aplicación web se diseñó para ser intuitiva y fácil de usar, con una interfaz gráfica que permite a los usuarios navegar por el mapa, visualizar las palmeras detectadas y acceder a la información detallada sobre cada una de ellas. El diagrama de secuencia de la figura 3.11 muestra el flujo de datos entre el backend y frontend. Actualmente la aplicación no cuenta con una base de datos, dado que las predicciones son *on-the-fly*, es decir, se realizan en el momento en que el usuario solicita la información. Esto permite reducir la complejidad del sistema y evitar la necesidad de mantener una base de datos adicional. Sin embargo, en futuras versiones se podría considerar su incorporación para almacenar las predicciones y mejorar el rendimiento del sistema, que actualmente se manejan en memoria del backend.

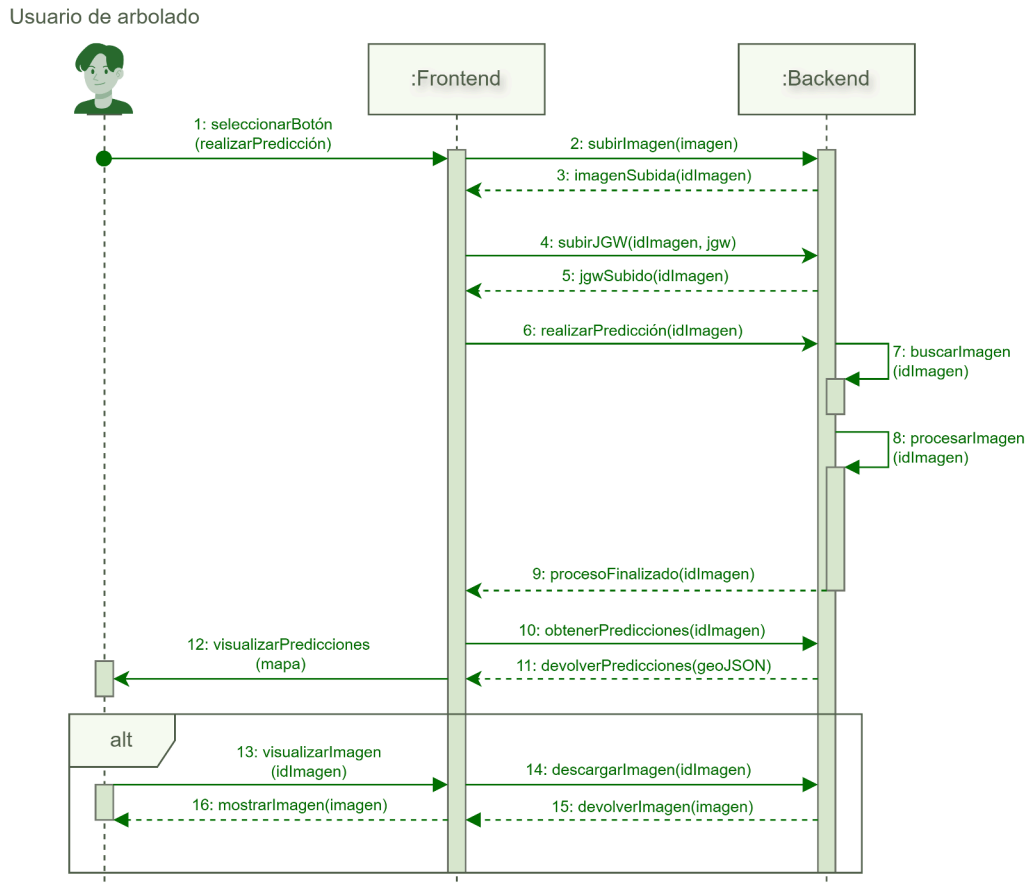


FIGURA 3.11. Diagrama de secuencia de la aplicación web Mis palmeras que muestra el flujo de datos entre el backend y frontend.

Un desafío importante en el desarrollo de la aplicación fue la integración con los modelos de detección del picudo rojo. Actualmente la integración está altamente acoplada (no se brinda como servicio separado, sino que se integra directamente en el backend). Esto implica que cualquier cambio en los modelos requiere una actualización del backend, lo que puede complicar el mantenimiento y la escalabilidad del sistema. Esta decisión respondió a criterios de agilidad en el desarrollo inicial del sistema al permitir una integración más rápida de los modelos. En futuras versiones, se podría considerar la implementación de una arquitectura basada en microservicios, en donde los modelos de detección se brinden como servicios independientes (por ejemplo, utilizando funciones *serverless*), lo que permitiría una mayor flexibilidad y escalabilidad del sistema.

Finalmente, en la figura 3.12 se puede observar una captura de pantalla de la página de inicio de la aplicación.

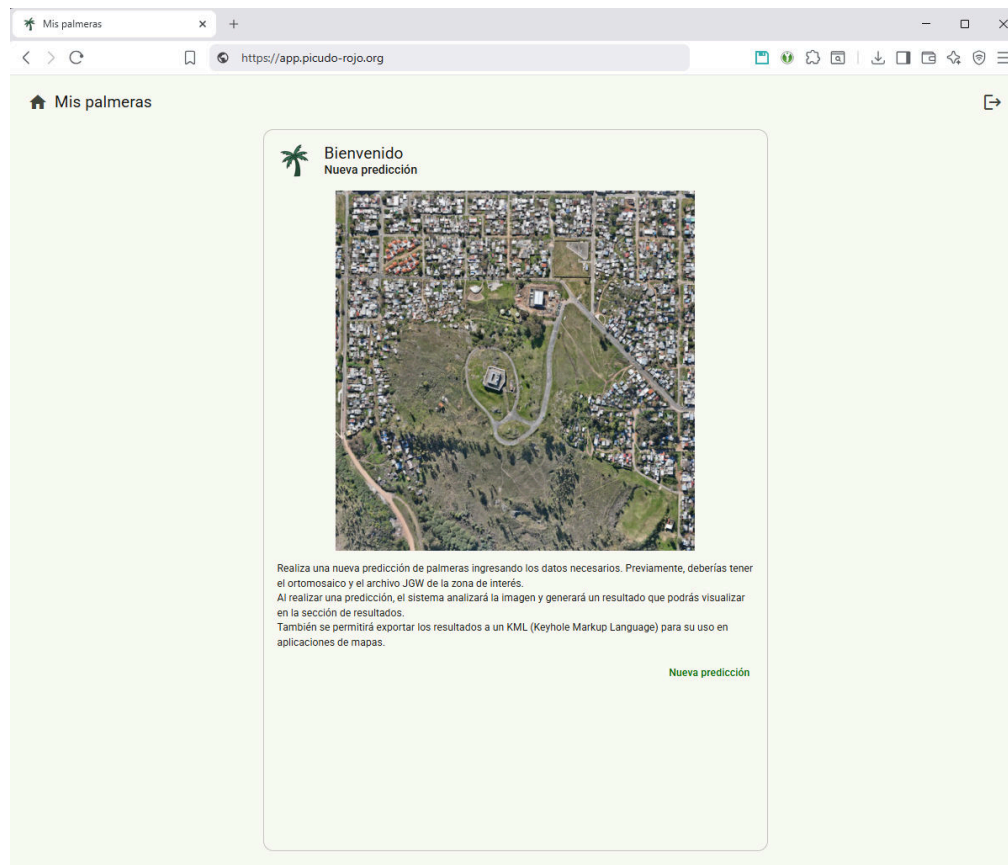


FIGURA 3.12. Captura de pantalla de la aplicación Mis Palmeras luego de iniciar sesión.

Capítulo 4

Ensayos y resultados

En este capítulo se describen los ensayos realizados y los resultados obtenidos en el desarrollo del sistema. Se detallan las etapas del entrenamiento de los modelos de detección y la integración de los modelos en la aplicación web, junto con su despliegue en producción.

4.1. Entrenamiento y resultados de modelos

Debido a la naturaleza del problema y a la poca cantidad de datos en las clases más importantes, se optó por realizar el entrenamiento de dos modelos: uno para la detección de palmeras y otro para la detección del picudo rojo. En base a la literatura estudiada (referirse a la sección 1.2), se sabía que modelos de la familia YOLO habían demostrado un buen desempeño en tareas de detección de palmeras en imágenes aéreas. Por lo tanto, se decidió utilizar YoloV11 (el modelo más actual de la familia YOLO hasta el momento de ejecución de los entrenamientos) como arquitectura base para ambos casos. La versión 11 tiene una mejora en performance y tiempo de procesamiento con respecto a versiones anteriores, como se puede apreciar en la figura 4.1.

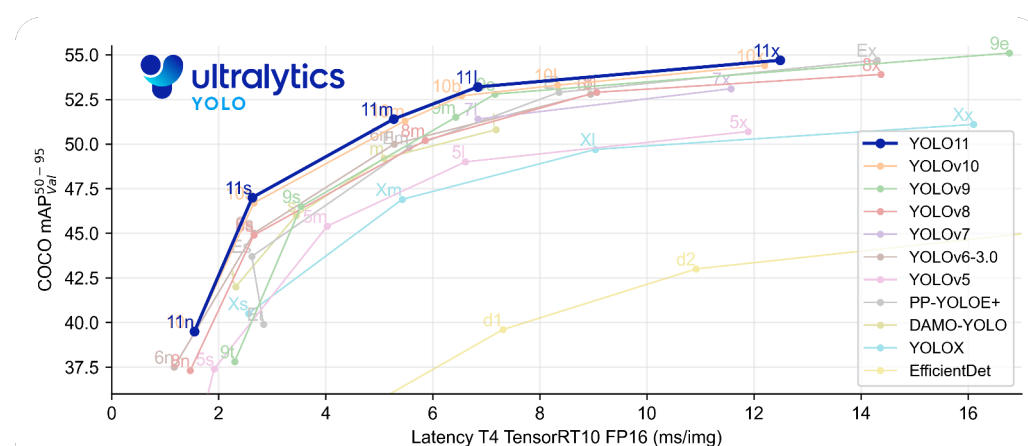


FIGURA 4.1. Comparación de performance entre YoloV11 y versiones anteriores ¹.

¹Imagen obtenida de <https://docs.ultralytics.com/es/models/yolo11/>

Según la evidencia existente se esperaba alcanzar un mAP superior al 0,85 (mAP50) en la detección de palmeras. Esto se basa principalmente en el artículo de N. Najiv Zhorif et al [12], donde se logró un mAP50 de cerca del 0,85 utilizando YoloV8 y SAHI, con imágenes de resoluciones similares.

El enfoque del entrenamiento se centró en el *fine-tuning* de este modelo base (en su versión *nano* y su versión extra grande) preentrenado en el dataset de COCO, utilizando *transfer learning* [77] para adaptar el modelo a la tarea específica de detección de palmeras. Luego, el modelo de detección de palmeras se utilizó como base para el entrenamiento del modelo de detección del picudo rojo, aplicando nuevamente *transfer learning*, con diferencia que en este caso se congeló las primeras capas del modelo (técnica obtenida de Yosinski et al. [78] y Goodfellow et al. [79]). Este enfoque puede visualizarse en la figura 4.2, que muestra la arquitectura oficial de YoloV11 y una simplificación de sus módulos (que contienen las capas de entrenamiento) congelados para este trabajo.

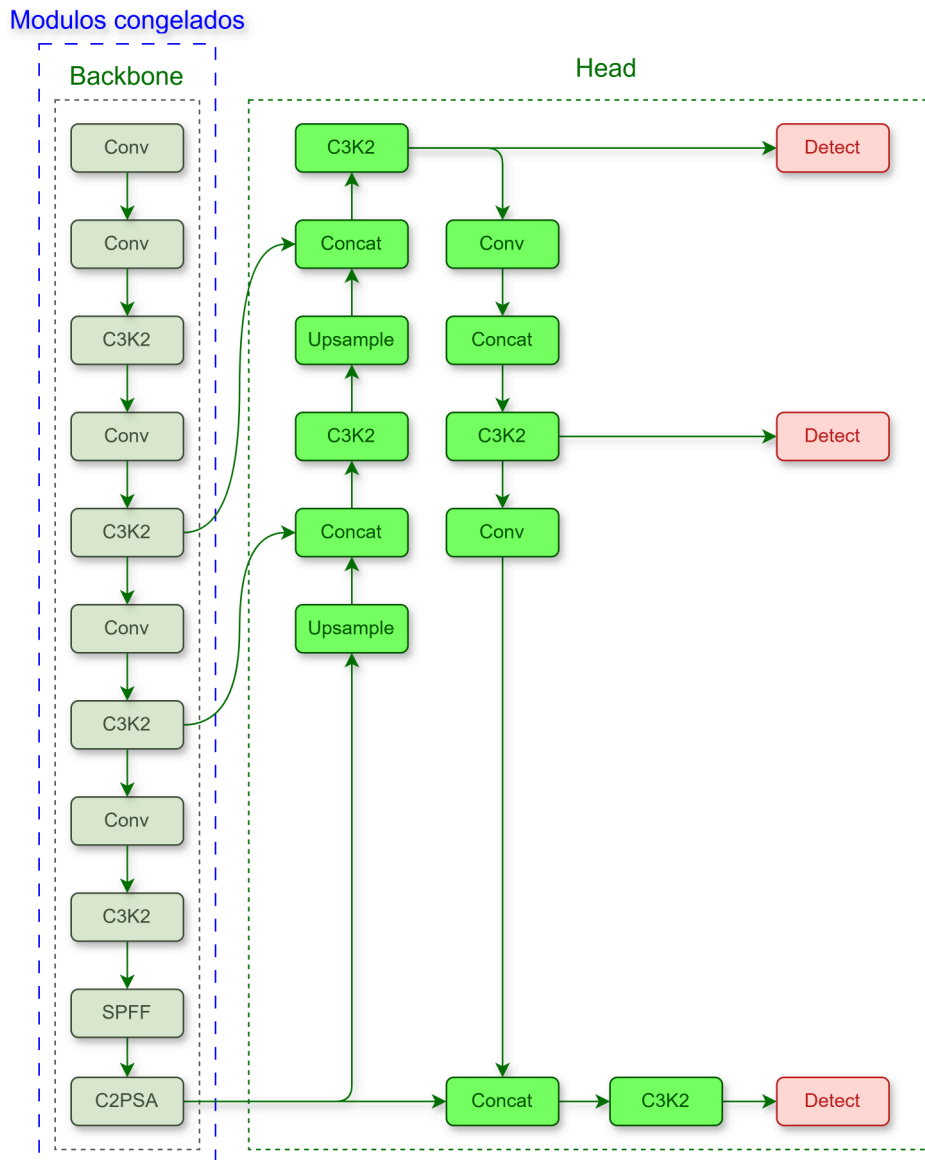


FIGURA 4.2. Esquema del enfoque de entrenamiento basado en *transfer learning*.

El banco de entrenamiento se dividió en dos infraestructuras principales: una utilizada para el entrenamiento del modelo de detección de palmeras y otra para el modelo de detección del picudo rojo. La primera, con un costo computacional más elevado, se basó en la infraestructura de Google Colab, en donde se contaba con una tarjeta gráfica NVIDIA A100 con 80 GB de memoria HBM2e. Se adaptaron los experimentos al uso de esta infraestructura, para aprovechar la gran capacidad de memoria para entrenar modelos y *batches* más grandes. La segunda infraestructura, utilizada principalmente para el entrenamiento del modelo de detección del picudo rojo, se basó en una estación de trabajo local que contaba una tarjeta gráfica NVIDIA GeForce RTX 4080 SUPER, con 16 GB de memoria GDDR6X. Esto permitió manejar la versión extra grande de YOLO sin problemas, tanto en la nube como en entorno local. Se utilizaron las bibliotecas de Ultralytics y PyTorch para la implementación y entrenamiento. Los hiperparámetros se ajustaron mediante experimentación, buscando un equilibrio entre la tasa de aprendizaje, el tamaño del *batch* y el número de épocas para evitar el sobreajuste. En todos los casos se utilizó *early stopping* para detener el entrenamiento cuando la métrica de validación no mejoraba después de un número determinado de épocas. Sin embargo, se hizo especial hincapié en la correcta selección de la cantidad de épocas para maximizar el desempeño del modelo. Un resumen del hardware utilizado para el entrenamiento se presenta en la tabla 4.1.

TABLA 4.1. Resumen del hardware utilizado para el entrenamiento de los modelos.

Componente	Especificación local	Especificación en la nube
GPU	NVIDIA GeForce RTX 4080 SUPER (16 GB GDDR6X)	NVIDIA A100 (80 GB HBM2e)
CPU	Intel Core i7-12700KF (12 núcleos)	Intel(R) Xeon(R) CPU @ 2.20GHz (12 núcleos)
RAM	128 GB DDR5 (5600 MHz)	167 GB
Almacenamiento	1 TB NVMe	1 TB NVMe
Sistema operativo	Windows 11	Ubuntu 24.04

4.1.1. Detección de palmeras

El primer modelo entrenado fue el de detección de palmeras, se utilizó el dataset procesado que contenía imágenes recortadas en subimágenes de 640×640 píxeles. Las transformaciones aplicadas desde el dataset *raw* al dataset *processed*, junto con la cantidad de imágenes o detecciones en cada caso, se resumen en la tabla 4.2.

TABLA 4.2. Resumen de las transformaciones aplicadas al dataset.

Etap	Descripción	Imágenes (train/val/test)	Detecciones (train/val/test)
1	Dataset original	Positivas: 1 288 Fondo: 0	Sanas: 8 403 Infectadas: 170 Muertas: 280 Exterminadas: 191
2	Eliminar palmeras exterminadas	-	Sanas: 8 403 Infectadas: 170 Muertas: 280 Exterminadas: 0
3	Unificar clases de palmeras	-	Palmeras: 8 853
4	Conversión a formato YOLO	-	-
5	Recortes de 640 × 640	Positivas: 14 863 Fondo: 91 565	Palmeras: 26 488
6	Balanceo de clases	Positivas: 14 863 Fondo: 1 486	-
7	<i>Splits</i> finales (80 %/10 %/10 %)	Positivas: 11 902 / 1 479 / 1 482 Fondo: 1 177 / 156 / 153	Palmeras: 21 280 / 2 585 / 2 623

Un punto importante a destacar en el *pipeline* de entrenamiento es que se eliminaron las palmeras exterminadas y se transformaron todas las palmeras infectadas, muertas y en buen estado en una sola clase denominada *palmera*, lo que resultó en un total de 16 349 imágenes con 26 488 palmeras. En la figura 4.3 se pueden observar ejemplos de imágenes del dataset procesado, que fueron utilizadas para el entrenamiento del modelo de detección de palmeras.

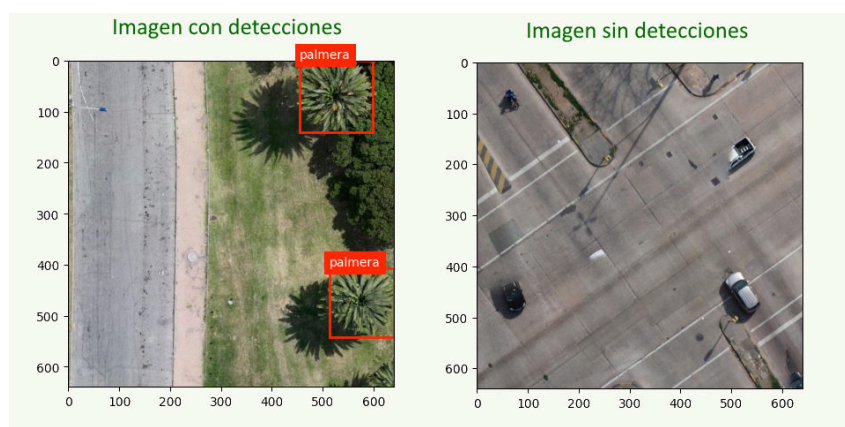


FIGURA 4.3. Ejemplos de imágenes del dataset procesado.

El tamaño total del dataset procesado fue de aproximadamente 1,21 GB. Este tamaño no incluye el aumento de datos, dado que se utilizó *online data augmentations* [80]. La gestión de las configuraciones de este aumento de datos se realizó mediante varios archivos en los que se definieron las transformaciones a aplicar durante el entrenamiento (en este caso, se utilizó directamente *Ultralytics* para el aumento de datos). Esto permitió un ahorro de espacio. Las transformaciones aplicadas fueron conservadoras, en el sentido de que no se modificaron las características esenciales de las palmeras, pero sí se aplicaron variaciones que podrían encontrarse en imágenes reales. Un ejemplo de las transformaciones aplicadas se puede observar en la figura 4.4.

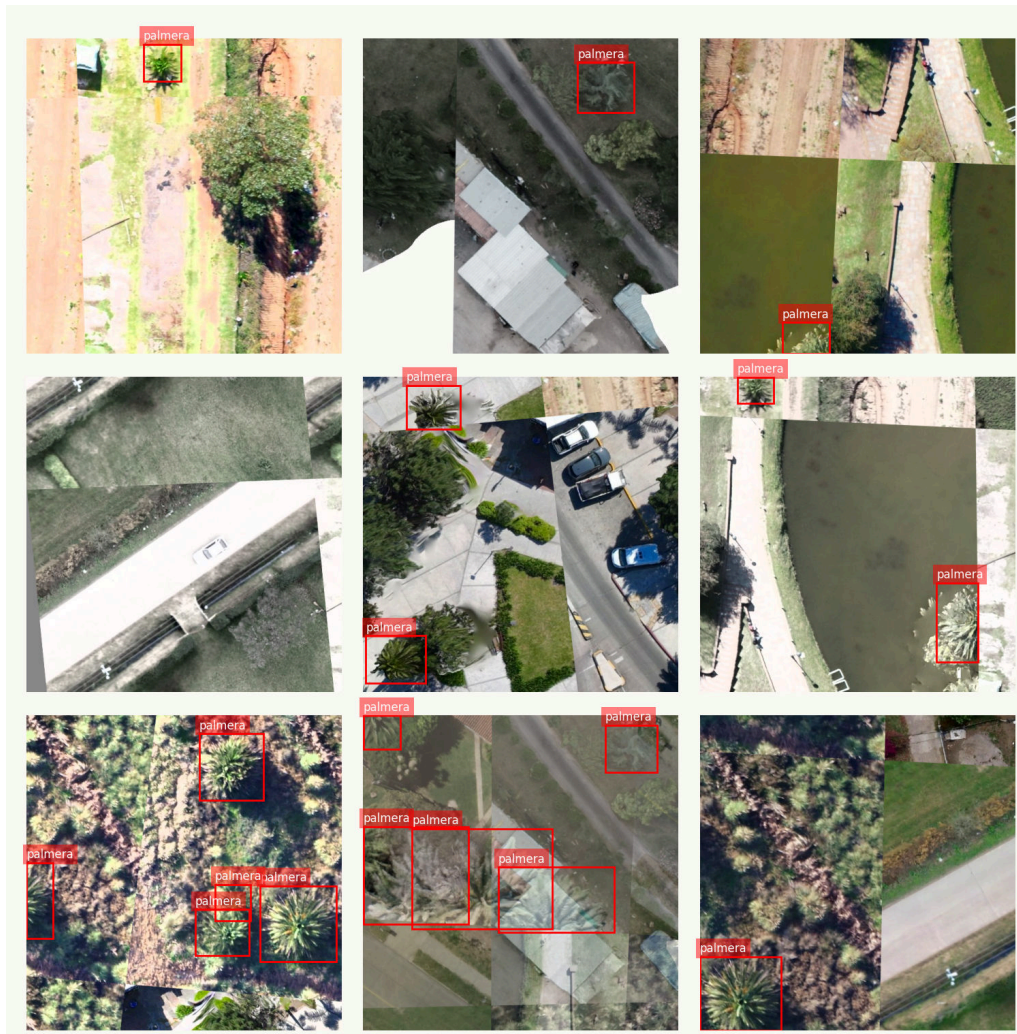


FIGURA 4.4. Ejemplo de transformaciones (aumento de datos) aplicadas al dataset de palmeras.

Se entrenaron dos versiones del modelo YoloV11: la versión *nano* (YoloV11n) y la versión extra grande (YoloV11x). La versión *nano* es más ligera y rápida, pero con menor capacidad de representación, mientras que la versión extra grande es más pesada y lenta, pero con mayor capacidad de representación. Para ello, se realizaron varios experimentos variando los hiperparámetros, como la tasa de aprendizaje, el tamaño del *batch*, el optimizador y el decaimiento de la tasa de

aprendizaje (esta prueba de hiperparámetros se realizó de forma manual, entrenando por menor cantidad de épocas y observando los resultados, usualmente con la versión *nano*). El mejor experimento para cada versión del modelo se resume en la tabla 4.3.

TABLA 4.3. Resumen de los mejores experimentos realizados para el entrenamiento del modelo de detección de palmeras.

Hiperparámetro	YoloV11n	YoloV11x
Infraestructura	Local	Nube
Épocas	300	300
Tamaño del <i>batch</i>	64	64
Tamaño de imagen	640	640
Aumento de datos	<i>Custom 1</i>	<i>Custom 1</i>
Optimizador	SGD	SGD
Tasa de aprendizaje	0,01	0,01
Decaimiento de la tasa	0,0005	0,0005
mAP50	0,977	0,984
mAP50-95	0,841	0,926
Precisión	0,956	0,970
Recall	0,943	0,966
F1-score	0,949	0,967
Tiempo medio por época	1 minuto	4 minutos
Tiempo total	5,5 horas	24 horas

La figura 4.5 muestra las curvas de entrenamiento del mejor experimento realizado con la versión extra grande del modelo YoloV11. Se puede observar que el modelo converge adecuadamente y no existe un gran sobreajuste a los datos. Concretamente, la pérdida en la caja comienza a sobreajuste sobre la época 210, pero se decidió continuar hasta la época 300 en donde sí comienza a divergir junto con la pérdida de clasificación. También se puede observar lo que parecería ser un sobreajuste significativo a partir de la época 290, pero este efecto es porque *Ultralytics* tiene la opción de cerrar el aumento de datos (para el caso del mosaico, que realiza una combinación de cuatro imágenes en una) en las últimas 10 épocas. Esto permite que el modelo se entrene en las imágenes originales sin estas transformaciones con el objetivo de mejorar la capacidad de generalización. Ajustar la cantidad de épocas en vez de dejar el entrenamiento hasta que ejecute un *early stopping* permite afinar el aprendizaje mediante una correcta especificación de la tasa de aprendizaje. En este caso se utilizó una tasa de aprendizaje con decaimiento lineal. Esto implica que al final del entrenamiento ésta sea muy baja, lo que permite un ajuste fino de los pesos del modelo. Este ajuste de épocas se estimó mediante experimentos sobre el modelo YoloV11n que al ser más liviano, permitió realizar pruebas más rápidas (usualmente 6 horas en la infraestructura local). En todo caso, el mAP50-95 seguía mejorando hasta la época 300.

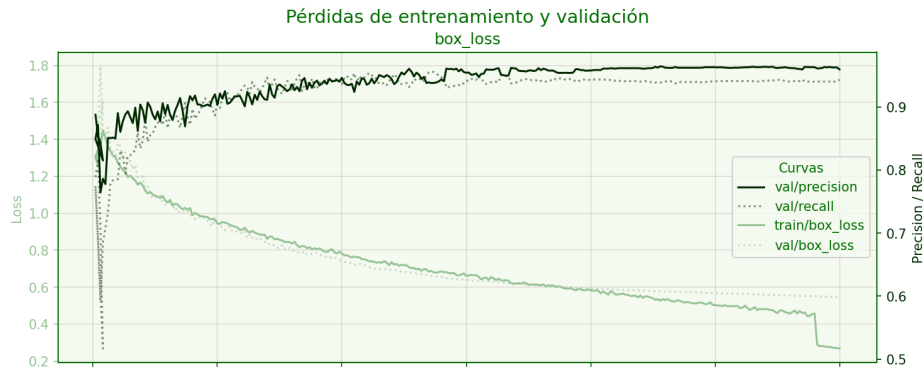
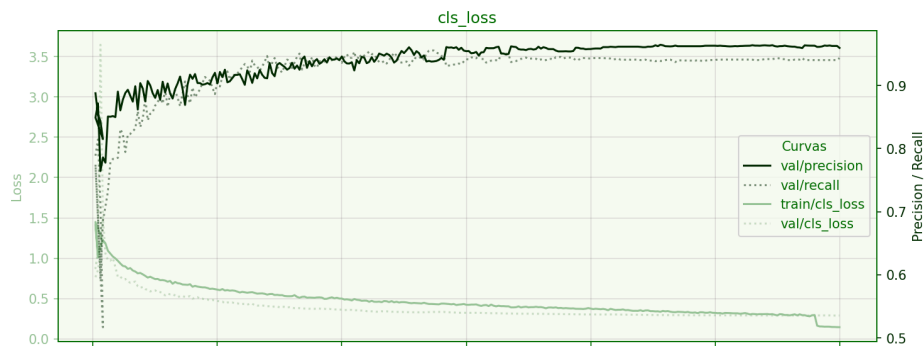
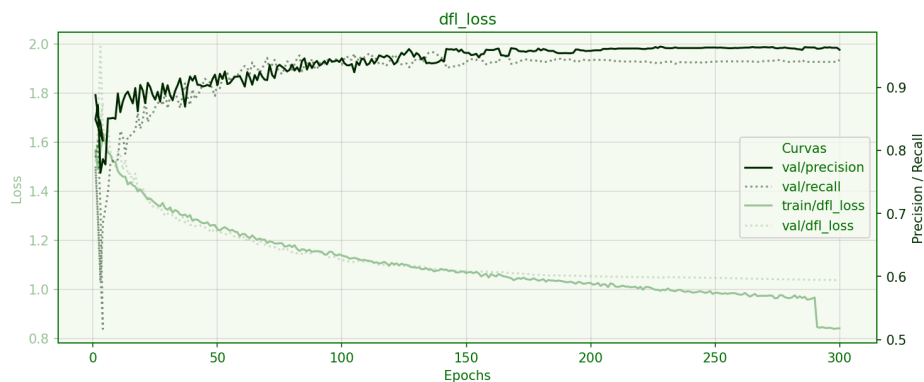
(A) Curva de pérdida de caja (*box loss*).(B) Curva de pérdida de clasificación (*cls loss*).(C) Curva de pérdida de focal (*dfl loss*).

FIGURA 4.5. Curvas de entrenamiento del mejor experimento realizado con YoloV11x para la detección de palmeras. En todas se sobrepone la precisión y recuperación para observar su evolución durante el entrenamiento.

El mejor experimento se entrenó en Google Colab (utilizando la versión Pro). En la época 100 ya se lograba un excelente desempeño, con un mAP50-95 de 0,84, lo que se observa en la figura 4.6. En este punto, el mAP50 ya había alcanzado un valor cercano al 0,97, la precisión un 0,94 y la recuperación un 0,95. Métricas muy altas, que indicaban que el modelo había aprendido a detectar las palmeras de manera efectiva. Sin embargo, dado que el objetivo era maximizar el desempeño del modelo para su uso en producción, se decidió continuar el entrenamiento hasta la época 300 (evitando un gran sobreajuste), para finalizar con un mAP50-95 de 0,926, una precisión de 0,970 y un recall de 0,966.

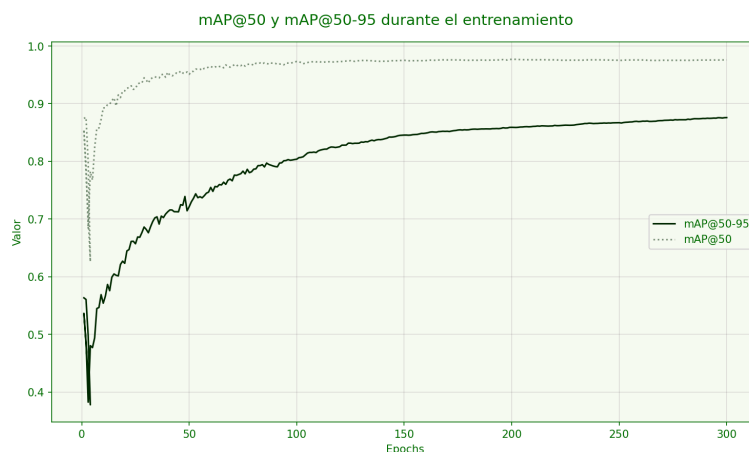


FIGURA 4.6. Curvas del mejor experimento realizado con YoloV11x para la detección de palmeras mostrando el mAP50-95 y mAP50 durante el entrenamiento sobre el conjunto de validación.

La matriz de confusión sobre el conjunto de *test*, que se puede observar en la figura 4.7, indica que el modelo tiene un buen desempeño en la detección de palmeras, con pocos falsos positivos y falsos negativos. En este caso una de las métricas más importantes fue la precisión, dado que se esperaba que se detectara la mayor cantidad de palmeras posibles mientras que los falsos positivos (detecciones erróneas) no eran tan críticos.

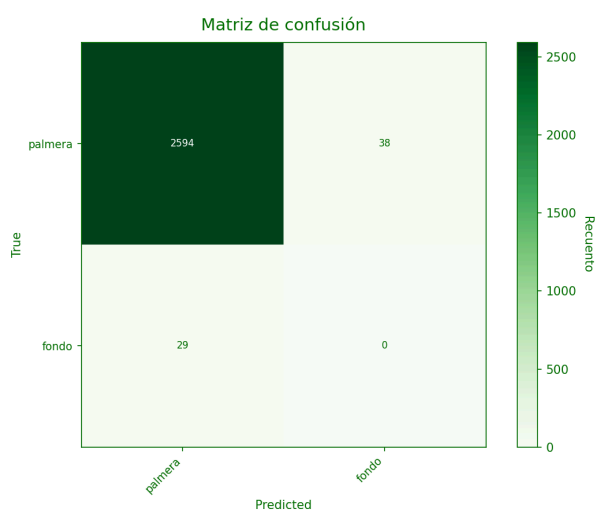


FIGURA 4.7. Matriz de confusión del mejor experimento realizado con YoloV11x para la detección de palmeras sobre el conjunto de *test*.

Como resumen final, el modelo YoloV11x entrenado para la detección de palmeras alcanzó un mAP50-95 de 0,973 (mAP50 de 0,993) en el conjunto de *test*, con un tiempo de entrenamiento aproximado de 1 día en la infraestructura cloud y un costo estimado de U\$S 40. Esto supera las expectativas iniciales dada la literatura revisada. Este modelo se seleccionó para su posterior integración en la aplicación web (con el objetivo de validar el funcionamiento de la web y mostrar avances) en comparación con la versión *nano*, que si bien es más rápida, no alcanzó un desempeño tan alto (el tiempo de inferencia no era tan crítico en este caso).

4.1.2. Detección del picudo rojo

Una vez entrenado y validado el modelo de detección de palmeras, se procedió a entrenar el modelo de detección del picudo rojo. Para ello se utilizó el modelo de detección de palmeras como base y se aplicó *transfer learning* para adaptar el modelo a la nueva tarea. A diferencia del caso anterior en el entrenamiento de este modelo se congelaron las capas iniciales. Específicamente se congeló el *backbone* (los primeros 11 módulos) y se dejó libre el *head* (resto de las capas). Esto permite preservar las características aprendidas en la detección de palmeras. Las transformaciones aplicadas fueron similares al de la sección anterior, con la diferencia que debido a la poca cantidad de clases para los casos de palmeras muertas e infectadas, se aplicó *undersampling*, como se mencionó en la sección 3.5. En la tabla 4.4 se resume la última transformación que dió función al dataset procesado.

TABLA 4.4. Resumen de las transformaciones aplicadas al dataset de picudo rojo durante el procesamiento.

Etapas	Descripción	Imágenes (train/val/test)	Detecciones (train/val/test)
7	<i>Splits</i> finales (80 %/10 %/10 %)	Positivas: 700/86/85 Fondo: 66/10/11	Palmeras sanas: 672/67/76 Palmeras infectadas: 422/53/46 Palmeras muertas: 389/43/55

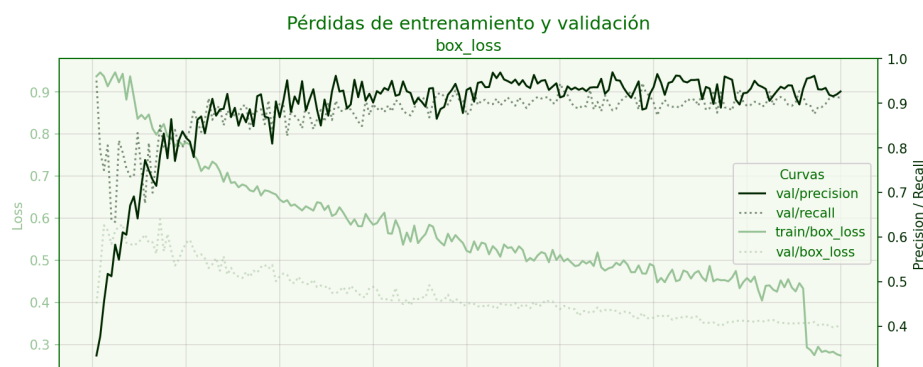
Las principales diferencias con respecto al dataset de palmeras son la cantidad de imágenes y detecciones, que en este caso son mucho menores. Las imágenes son las mismas, solo que ahora se encuentran etiquetadas con las clases correspondientes. Se puede referir a la figura 4.3 para observar ejemplos de imágenes del dataset procesado (con la diferencia en las clases de las etiquetas) y se puede referir a la figura 3.7 para observar ejemplos específicos de imágenes con palmeras infectadas y muertas. El tamaño total del dataset procesado fue de aproximadamente 1,21 GB. Nuevamente se utilizó *online data augmentations* para el manejo del aumento de datos, aplicando transformaciones similares al caso anterior.

A diferencia de la sección anterior, en este caso, dada la poca cantidad de datos, se decidió realizar una búsqueda más exhaustiva de los hiperparámetros, utilizando técnicas de *hyperparameter tuning* [81] para encontrar la configuración óptima. Se entrenaron las dos versiones del modelo YoloV11: la versión *nano* (YoloV11n) y la versión extra grande (YoloV11x). El mejor experimento para cada versión del modelo se resume en la tabla 4.5.

TABLA 4.5. Resumen de los mejores experimentos del entrenamiento del modelo de detección del picudo rojo.

Hiperparámetro	YoloV11n	YoloV11x
Épocas	200	200
Mejor época	171	197
Tamaño del <i>batch</i>	64	16
Tamaño de imagen	640	640
Aumento de datos	<i>Custom 4</i>	<i>Custom 4</i>
Optimizador	AdamW	AdamW
Tasa de aprendizaje	0,001429	0,001429
Decaimiento de la tasa	0,0005	0,0005
mAP50	0,922	0,967
mAP50-95	0,848	0,926
Precisión	0,862	0,918
Recall	0,851	0,910
F1-score	0,856	0,913
Tiempo medio por época	3 segundos	10 segundos
Tiempo total	20 minutos	1 hora

En la figura 4.8 se pueden observar las curvas de entrenamiento y validación del mejor experimento realizado con la versión extra grande del modelo YoloV11. En este caso se observa una mayor varianza dado que la cantidad de datos es mucho menor. Para la estimación de las épocas finales, dado que el costo en tiempo no era tan elevado, se decidió entrenar inicialmente por 300 épocas e identificar si el modelo comenzaba a sobreajustar o se detenía de forma anticipada mediante un *early stopping* de 50 épocas. El entrenamiento se detuvo en la época 235, en donde el mejor modelo fue en la época 198. Finalmente, se entrenó nuevamente hasta la época 200 (con el objetivo de cerrar el aumento de datos 10 épocas antes del final) para obtener el modelo final.

(A) Curva de pérdida de caja (*box loss*).

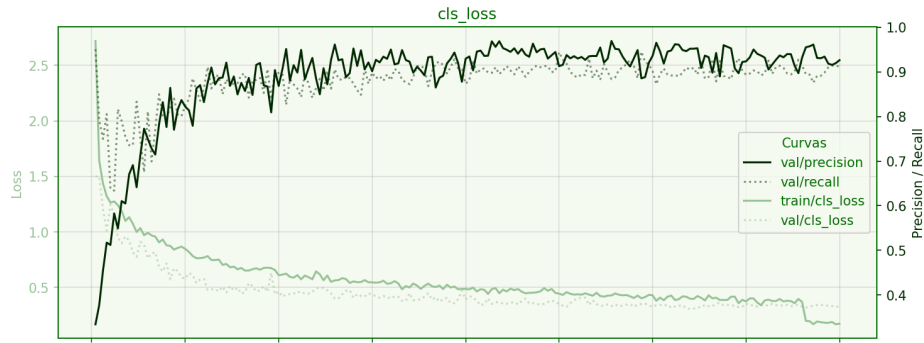
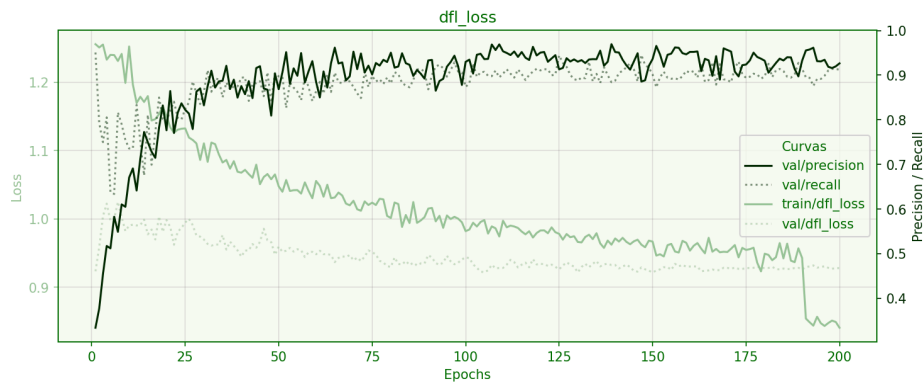
(B) Curva de pérdida de clasificación (*cls loss*).(C) Curva de pérdida de focal (*dfl loss*).

FIGURA 4.8. Curvas de entrenamiento del mejor experimento realizado con YoloV11x para la detección del picudo rojo. En todas se sobrepone la precisión y recuperación para observar su evolución durante el entrenamiento.

Estos experimentos se realizaron totalmente en la infraestructura local, dado que el tamaño del dataset era manejable. El hecho de congelar las primeras capas permitió reducir el tiempo de entrenamiento significativamente dado que este congelamiento reduce la cantidad de parámetros optimizables. Concretamente, se redujo la cantidad de parámetros entrenables de aproximadamente 56 millones a 28 millones, o sea, un 50 % del total de parámetros del modelo. Esto permitió entrenar el modelo en aproximadamente 1 hora, en comparación con las 12 horas que hubiera tomado entrenar todas las capas.

Con respecto a las métricas de validación, en la época 100 ya se lograba un desempeño aceptable, con un mAP50-95 de 0,89, lo que se observa en la figura 4.9. En este punto, el mAP50 ya había alcanzado un valor cercano al 0,951, la precisión un 0,93 y la recuperación un 0,874. Si bien las métricas eran buenas, dada la gran variabilidad en el aprendizaje, se decidió continuar el entrenamiento hasta la última época nombrada anteriormente, para finalizar con un mAP50-95 de 0,926, una precisión de 0,919 y un recall de 0,91, en la época 197.

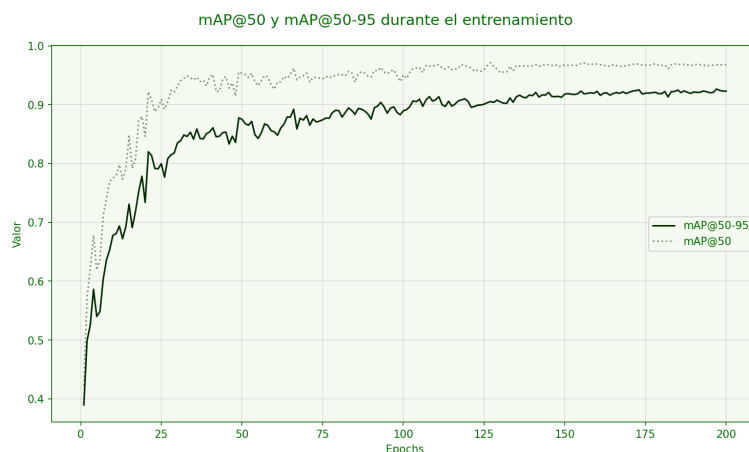


FIGURA 4.9. Curvas del mejor experimento realizado con YoloV11x para la detección del picudo rojo mostrando el mAP50-95 y mAP50 durante el entrenamiento sobre el conjunto de validación.

Finalmente, la matriz de confusión (sobre el conjunto de *test*) de la figura 4.10 indica que el modelo tiene un buen desempeño en la detección del picudo rojo. Sin embargo, estos datos deben ser tomados con cautela, dado que la cantidad de datos es muy limitada y podría no representar adecuadamente la variabilidad del mundo real.

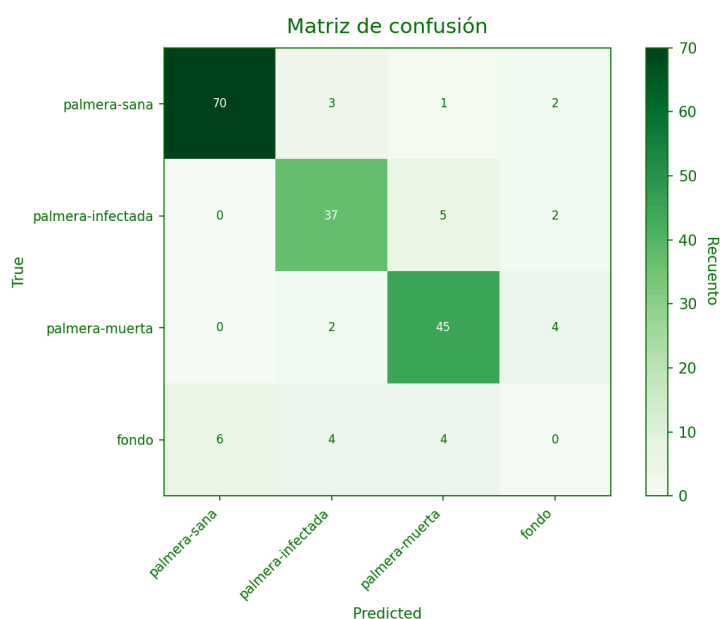


FIGURA 4.10. Matriz de confusión del mejor experimento realizado con YoloV11x para la detección del picudo rojo sobre el conjunto de *test*.

Como resumen final, el modelo YoloV11x entrenado para la detección del picudo rojo alcanzó un mAP50-95 de 0,869 (mAP50 de 0,928) en el conjunto de *test*, que es un buen resultado dado la poca cantidad de datos disponibles. Se debe destacar que también se probó entrenar el modelo YoloV11x directamente con todas las capas libres (lo que tomó cerca de 12 horas en la infraestructura local), pero los

resultados no fueron fructíferos. El modelo final resultante se seleccionó para su posterior integración en la aplicación web.

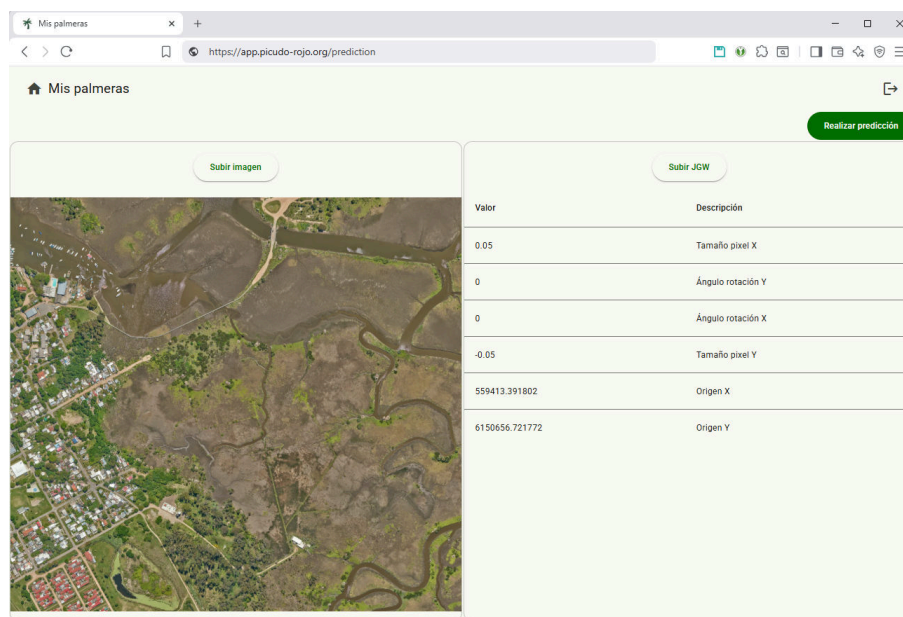
4.2. Integración del modelo en la aplicación web

La integración de los modelos entrenados en la aplicación web se realizó utilizando FastAPI para crear *endpoints* que permitan la inferencia de los modelos. Se creó un *endpoint* para cada modelo: uno para la detección de palmeras y otro para la detección del picudo rojo aunque inicialmente el objetivo era la detección del picudo rojo, varios grupos encargados de la gestión de la plaga indicaron sus necesidades de reconocer palmeras, por lo que se les brindó también esta funcionalidad.

Estos *endpoints* reciben imágenes en formato base64, las procesan y devuelven las predicciones en formato GeoJSON [82] (simulando el flujo expuesto en figura 3.11). Luego, se muestran en un mapa generado con una aplicación en Angular que utiliza la biblioteca Leaflet [83], desde donde también se permite descargar la imagen para una mejor visualización. En el backend, la integración se hizo simplemente encapsulando las funcionalidades auxiliares (creando *wheels* livianas de Python) utilizadas durante el trabajo, en donde se incluyeron únicamente las dependencias obligatorias. No se utilizó una separación entre la capa de aplicación y la capa de predicción, sino que directamente se integró *Ultralytics* en FastAPI.

Para la predicción se utilizó la biblioteca *Supervision* [84], que permite la aplicación de SAHI de forma simple, configurar varios parámetros necesarios como NMS (*Non-Maximum Suppression* [85]) y que brinda una excelente anotación sobre las imágenes.

Desde el punto de vista del usuario, inicialmente se sube la imagen y sus coordenadas en formato .jgw, como se puede apreciar en la siguiente figura 4.11.



Valor	Descripción
0.05	Tamaño pixel X
0	Ángulo rotación Y
0	Ángulo rotación X
-0.05	Tamaño pixel Y
559413.391802	Origen X
6150656.721772	Origen Y

FIGURA 4.11. Captura de pantalla de la carga de datos, en donde se sube una imagen y sus coordenadas asociadas.

Luego para realizar el análisis el usuario selecciona el botón realizar predicción. En este punto, el frontend realiza varios pasos de forma sincrónica. Inicialmente, realiza un POST al *endpoint* `/apiv1/image/upload` para subir la imagen y otro POST al *endpoint* `/apiv1/jgw/upload/{image_id}` para subir la información de las coordenadas, con el identificador de la imagen devuelto en la petición anterior. Luego de que está subida la imagen (se guarda en memoria con FastAPI) y su georreferenciación, se ejecuta un POST al *endpoint* `/apiv1/predictions/generate_sync_prediction/{image_id}` que inicia el proceso de predicción sobre dicha imagen de forma sincrónica. En este punto, el backend realiza la predicción sobre la imagen, convierte los resultados a GeoJSON y se los envía al frontend. El frontend renderiza los puntos en el mapa y les adjunta información de referencia (como si está infectada la palmera o no). Finalmente, el usuario tiene la posibilidad de descargar la imagen anotada. Para esto, el frontend realiza una solicitud GET al siguiente *endpoint* `/apiv1/image/downloadAnnotated/{image_id}` para obtener la imagen anotada (generada en el momento por el backend). En la figura 4.12 se puede apreciar el mapa con las predicciones sobre la zona del mirador de Santiago Vazquez (con el identificador K-29-A-6-N-3 en el SIG). En la figura de la izquierda, el mapa georreferenciado y en la figura de la derecha la imagen devuelta con las cajas (*bounding boxes*) sobre las palmeras detectadas en donde el color violeta indica palmeras sanas, el color naranja palmeras infectadas y el color rojo palmeras muertas.



(A) Captura de pantalla del mapa con las predicciones.

(B) Captura de pantalla de la imagen (ortomosaico recortado) anotada con las predicciones.

FIGURA 4.12. Ejemplo de análisis una imagen en la aplicación web.

Para la inferencia, se utilizó SAHI para manejar las imágenes grandes y ortomosaicos (por medio de *Supervision*) con un *overlapping* de 0,1 (un 10 % de la imagen) y un NMS con un *threshold* de 0,9. Sin embargo, la predicción por este medio implica varios desafíos adicionales como el manejo de las superposiciones entre recortes y la unificación de las predicciones, que no se solucionan con técnicas clásicas como NMS o técnicas como *NoN Maximum Merge* [86]. Este tema se abordó con un enfoque combinado de varias técnicas que se listan en la tabla 4.6.

TABLA 4.6. Técnicas aplicadas en el posprocesamiento de las predicciones del modelo de detección del picudo rojo.

Fase	Técnica	Descripción
1	NMS	Aplicación de NMS con un <i>threshold</i> de 0,9.
2	Filtrado por <i>square ratio</i>	Filtrado por relación de aspecto cuadrado con un <i>threshold</i> de 0,7.
3	NMS (agnóstico)	Aplicación de NMS agnóstico a las clases con un <i>threshold</i> de 0,1.
4	Filtrado por <i>containment</i>	Filtrado de predicciones que están contenidas dentro de otras con un <i>threshold</i> de 0,8.
5	Filtrado por confianza	Filtrado final por confianza con un <i>threshold</i> de 0,5.

Al ser las imágenes de grandes ortomosaicos y el procesamiento mediante SAHI el tiempo de inferencia es alto, pero esto no es un problema dado que el objetivo no es la utilización en tiempo real de la aplicación. Se ha validado con imágenes de hasta 350 MB en donde el tiempo de inferencia fue de aproximadamente 5 minutos. Como métrica final, se obtuvo un ratio de 1 minuto por kilómetro cuadrado de tiempo de predicción (para imágenes de 5 cm de resolución espacial). Esto implica que se necesitaría aproximadamente 15 horas para predecir sobre todo Montevideo (utilizando, por ejemplo, el vuelo del año 2024, que cuenta con aproximadamente 871 ortomosaicos de $0,75 \text{ km}^2$ cada uno) en la infraestructura local.

Capítulo 5

Conclusiones

En el presente capítulo se exponen las conclusiones generales del trabajo realizado, se destacan los logros alcanzados y se reflexiona sobre los desafíos enfrentados. Además, se delinean los próximos pasos a seguir para continuar el desarrollo y mejora continua de la plataforma.

5.1. Conclusiones generales

La implementación de este sistema de detección basado en aprendizaje profundo ha demostrado ser una forma efectiva para abordar el problema de la identificación del picudo rojo en imágenes aéreas de palmeras. Si bien el objetivo inicial era enfocarse en la detección temprana, durante el trabajo se identificó que este problema es complejo y aún no resuelto, sin embargo, se podía aportar valor en una detección tardía a gran escala. Este sistema permite mejorar la gestión de la plaga y reducir los costos asociados al monitoreo manual, al mismo tiempo que contribuye a la preservación del arbolado público.

Desde el punto de vista de la IM, este trabajo marca un hito importante en la aplicación de tecnologías avanzadas en la agricultura urbana, lo que la posiciona como uno de los referentes en el uso de inteligencia artificial para la protección del arbolado público. La aplicación de estas tecnologías construye hacia una ciudad más inteligente y sostenible, alineada con las tendencias globales de *smart cities*.

Desde el punto de vista académico, este trabajo es pionero en la detección tardía del picudo rojo, no solo en la región, sino también a nivel mundial. La combinación de técnicas de visión por computadora y aprendizaje profundo aplicada a imágenes aéreas representa un avance en el campo de la teledetección y la observación terrestre (*Earth Observation*).

A lo largo del trabajo, se lograron varios hitos importantes:

- Se desarrolló y entrenó un modelo de detección del picudo rojo utilizando la arquitectura YoloV11, adaptándola a las características específicas del dominio.
- Se implementó técnicas de *data augmentation* y *transfer learning* para mejorar el rendimiento del modelo a pesar de la limitada disponibilidad de datos.
- Se diseñó una arquitectura de sistema robusta y escalable, que permite la integración de nuevos datos y la actualización del modelo de manera eficiente.

- Se validaron herramientas y frameworks *open source* alineados con las tecnologías actuales presentes en la IM.
- Se establecieron métricas claras de evaluación, como el mAP y la matriz de confusión, que permitieron un análisis detallado del desempeño del modelo.
- Se logró un mAP50-95 superior al 0,85 en la detección del picudo rojo, lo que cumple con los objetivos iniciales del proyecto.
- Se integró el modelo en una aplicación web y se lo disponibilizó al sector de arbolado de la IM.

Más allá de los logros técnicos, este trabajo ha sentado las bases para futuros proyectos, investigaciones y desarrollos en el campo de la visión por computadora. Se espera que los resultados obtenidos sirvan como referencia para otros proyectos similares dentro de la organización y en el ámbito académico.

Finalmente, destacar los desafíos enfrentados durante el desarrollo del trabajo, como la correcta gestión de la calidad de los datos, la necesidad de soluciones a medida en dominios específicos y la integración del sistema en un entorno real. Estos desafíos han sido superados con éxito, lo que demostró la viabilidad y efectividad del enfoque adoptado.

5.2. Próximos pasos

A partir de los logros alcanzados, se identifican varias áreas para continuar el trabajo en el futuro:

- Integrar datos de técnicas de detección temprana del picudo rojo, como dispositivos sonoros o sensores químicos, para complementar la detección visual.
- Incorporar técnicas de autoetiquetado que no pudieron ser implementadas en esta etapa.
- Ampliar la base de datos de imágenes para incluir más variabilidad principalmente en las clases palmera muerta e infectada.
- Adaptar otros módulos al sistema, como el de reportes.
- Desarrollar un plan de mantenimiento y actualización del modelo que contemple la incorporación de nuevos datos y la adaptación a cambios en el entorno.
- Extender la detección a todo el país, aprovechando los recursos de entidades nacionales como el Ministerio de Ganadería, Agricultura y Pesca (MGAP) y el Ministerio de Ambiente.

Bibliografía

- [1] Anticimex. *Picudo Rojo | Daños y tratamientos - Anticimex*. URL: <https://www.anticimex.es/picudo-rojo/> (visitado 21-03-2025).
- [2] A. Poplin et al. *Palm Weevils*. Mayo de 2014.
- [3] MGAP. «Información actualizada sobre el picudo rojo de las palmeras a setiembre 2024». En: (). URL: <https://www.gub.uy/ministerio-ganaderia-agricultura-pesca/comunicacion/noticias/informacion-actualizada-sobre-picudo-rojo-palmeras-setiembre-2024> (visitado 21-03-2025).
- [4] Alfonso Arcos. *PICUDO ROJO (Rynchophorus ferrugineus) EN MONTEVIDEO MONITOREO Y CONTROL (Avances)*. Montevideo, ago. de 2024. (Visitado 21-03-2025).
- [5] Intendencia de Montevideo. *Acciones de la Intendencia de Montevideo contra el Picudo Rojo | Portal institucional*. URL: <https://montevideo.gub.uy/noticias/urbanismo-y-obras/acciones-de-la-intendencia-de-montevideo-contr-a-el-picudo-rojo> (visitado 26-03-2025).
- [6] Pedro Hernández Sánchez. «Cirugía especializada en palmeras». es. En: ().
- [7] Mingliang Gao et al. «Recent Advances in Computer Vision: Technologies and Applications». en. En: *Electronics* 13.14 (ene. de 2024). Number: 14 Publisher: Multidisciplinary Digital Publishing Institute, pág. 2734. ISSN: 2079-9292. DOI: [10.3390/electronics13142734](https://doi.org/10.3390/electronics13142734). URL: <https://www.mdpi.com/2079-9292/13/14/2734> (visitado 21-03-2025).
- [8] Manav Sutar. «Convolutional Neural Networks (CNNs): Advancements and Future Trends». En: *Convolutional Neural Networks (CNNs): Advancements and Future Trends* (ene. de 2025). URL: https://www.academia.edu/128256115/Convolutional_Neural_Networks_CNNs_Advancements_and_Future_Trends (visitado 21-03-2025).
- [9] Joseph Redmon et al. *You Only Look Once: Unified, Real-Time Object Detection*. arXiv:1506.02640 [cs]. Mayo de 2016. DOI: [10.48550/arXiv.1506.02640](https://doi.org/10.48550/arXiv.1506.02640). URL: <http://arxiv.org/abs/1506.02640> (visitado 21-03-2025).
- [10] Dima Kagan, Galit Fuhrmann Alpert y Michael Fire. «Automatic large scale detection of red palm weevil infestation using street view images». en. En: *ISPRS Journal of Photogrammetry and Remote Sensing* 182 (dic. de 2021), págs. 122-133. ISSN: 09242716. DOI: [10.1016/j.isprsjprs.2021.10.004](https://doi.org/10.1016/j.isprsjprs.2021.10.004). URL: <https://linkinghub.elsevier.com/retrieve/pii/S0924271621002665> (visitado 21-11-2024).
- [11] Shaoqing Ren et al. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. arXiv:1506.01497 [cs]. Ene. de 2016. DOI: [10.48550/arXiv.1506.01497](https://doi.org/10.48550/arXiv.1506.01497). URL: <http://arxiv.org/abs/1506.01497> (visitado 21-03-2025).
- [12] Naufal Najiv Zhorif et al. «Implementation of Slicing Aided Hyper Inference (SAHI) in YOLOv8 to Counting Oil Palm Trees Using High-Resolution Aerial Imagery Data». en. En: *International Journal of Advanced Computer Science and Applications* 15.7 (2024). ISSN: 21565570, 2158107X. DOI: [10.145](https://doi.org/10.145)

- 69/IJACSA.2024.0150786. URL: <http://thesai.org/Publications/ViewPaper?Volume=15&Issue=7&Code=ijacsa&SerialNo=86> (visitado 22-03-2025).
- [13] M. S. Muna et al. «Development of Automatic Counting System for Palm Oil Tree Based on Remote Sensing Imagery». En: *Proceedings of the International Conference on Sustainable Environment, Agriculture and Tourism (ICOSEAT 2022)*. Atlantis Press, ene. de 2023. DOI: [10.2991/978-94-6463-086-2_68](https://doi.org/10.2991/978-94-6463-086-2_68).
- [14] H. Wibowo et al. «Large-Scale Oil Palm Trees Detection from High-Resolution Remote Sensing Images Using Deep Learning». En: *Big Data and Cognitive Computing* 6.3 (sep. de 2022). DOI: [10.3390/bdcc6030089](https://doi.org/10.3390/bdcc6030089).
- [15] Adel Ammar, Anis Koubaa y Bilel Benjdira. «Deep-Learning-Based Automated Palm Tree Counting and Geolocation in Large Farms from Aerial Geotagged Images». en. En: *Agronomy* 11.8 (jul. de 2021), pág. 1458. ISSN: 2073-4395. DOI: [10.3390/agronomy11081458](https://doi.org/10.3390/agronomy11081458). URL: <https://www.mdpi.com/2073-4395/11/8/1458> (visitado 22-03-2025).
- [16] D. P. T. Wardana, R. S. Sianturi y R. Fatwa. «Detection of Oil Palm Trees Using Deep Learning Method with High-Resolution Aerial Image Data». En: *ACM International Conference Proceeding Series*. Association for Computing Machinery, 2023, págs. 90-98. DOI: [10.1145/3626641.3626667](https://doi.org/10.1145/3626641.3626667).
- [17] D. Sandya Prasvita, D. Chahyati y A. M. Arymurthy. *Automatic Detection of Oil Palm Growth Rate Status with YOLOv5*.
- [18] Y. Nuwara, W. K. Wong y F. H. Juwono. «Modern Computer Vision for Oil Palm Tree Health Surveillance using YOLOv5». En: *2022 International Conference on Green Energy, Computing and Sustainable Technology, GECOST 2022*. Institute of Electrical y Electronics Engineers Inc., 2022, págs. 404-409. DOI: [10.1109/GECOST55694.2022.10010668](https://doi.org/10.1109/GECOST55694.2022.10010668).
- [19] M. H. Junos et al.
- [20] Stephanie Delalieux et al. «Red Palm Weevil Detection in Date Palm Using Temporal UAV Imagery». en. En: *Remote Sensing* 15.5 (ene. de 2023). Number: 5 Publisher: Multidisciplinary Digital Publishing Institute, pág. 1380. ISSN: 2072-4292. DOI: [10.3390/rs15051380](https://doi.org/10.3390/rs15051380). URL: <https://www.mdpi.com/2072-4292/15/5/1380> (visitado 21-03-2025).
- [21] Intendencia de Montevideo. *Sistema de Información Geográfica*. es. URL: <http://sig.montevideo.gub.uy/> (visitado 26-03-2025).
- [22] Izham Mokhtar et al. *Image Processing Systems for Unmanned Aerial Vehicle: State-of-the-art*. Ago. de 2023. DOI: [10.20944/preprints202308.1855.v1](https://doi.org/10.20944/preprints202308.1855.v1).
- [23] Unmanned System technology. *UAV/Drone Cameras for Commercial, Industrial, and Military Applications*. en-US. URL: <https://www.unmannedsystemstechnology.com/expo/cameras/> (visitado 27-03-2025).
- [24] 2000 Aviation. *Ortoimágenes – 2000 Aviation*. es. URL: <https://www.2000aviation.com/ortoimagenes/> (visitado 27-03-2025).
- [25] DJI. *Support for Phantom 4 Pro*. en. URL: <https://www.dji.com/global/support/product/photo> (visitado 27-03-2025).
- [26] DJI. *Specs - DJI Mavic 3 Enterprise - DJI Enterprise*. en. URL: <https://enterprise.dji.com/mavic-3-enterprise/photo> (visitado 27-03-2025).
- [27] Wikipedia. *RTK (navegación)*. es. Page Version ID: 161740605. Ago. de 2024. URL: [https://es.wikipedia.org/w/index.php?title=RTK_\(navegaci%C3%B3n\)&oldid=161740605](https://es.wikipedia.org/w/index.php?title=RTK_(navegaci%C3%B3n)&oldid=161740605) (visitado 27-03-2025).
- [28] ComNav Technology Ltd. *Receptor GNSS T300 Plus-Receptor GNSS-ComNav Technology Ltd*. zh-CN. URL: <https://www.comnavtech.com/sp/product/receiver/T300Plus.html> (visitado 28-03-2025).

- [29] Wikipedia. *Networked Transport of RTCM via Internet Protocol*. en. Page Version ID: 1277712218. Feb. de 2025. URL: https://en.wikipedia.org/w/index.php?title=Networked_Transport_of_RTCM_via_Internet_Protocol&oldid=1277712218 (visitado 27-03-2025).
- [30] Ian Sommerville. *Software Engineering*. Addison-Wesley, mar. de 2015.
- [31] Dirk Merkel. «Docker: lightweight Linux containers for consistent development and deployment». En: *Linux journal* 2014.239 (mar. de 2014), págs. 2-. URL: <https://dl.acm.org/citation.cfm?id=2600239.2600241>.
- [32] Docker. *Docker documentation*. en. 0. URL: <https://docs.docker.com/> (visitado 27-03-2025).
- [33] HashiCorp. *Vagrant*. 2023.
- [34] HashiCorp. *Documentation | Vagrant | HashiCorp Developer*. en. URL: <https://developer.hashicorp.com/vagrant/docs> (visitado 27-03-2025).
- [35] Wikipedia. *Protocolo ligero de acceso a directorios*. es. Page Version ID: 162588716. Sep. de 2024. URL: https://es.wikipedia.org/w/index.php?title=Protocolo_ligero_de_acceso_a_directorios&oldid=162588716 (visitado 27-03-2025).
- [36] LDAP Authors. *LDAP: Light LDAP implementation*. 2023.
- [37] Cloud Native Computing Foundation. *Keycloak*. en. URL: <https://www.keycloak.org/> (visitado 30-08-2025).
- [38] Boris Sekachev et al. *opencv/cvat: v1.1.0*. Ago. de 2020. DOI: [10.5281/zenodo.4009388](https://doi.org/10.5281/zenodo.4009388). URL: <https://doi.org/10.5281/zenodo.4009388>.
- [39] B. E. Moore y J. J. Corso. «FiftyOne». En: *GitHub*. Note: <https://github.com/voxel51/fiftyone> (2020).
- [40] MinIO, Inc. *MinIO: High Performance Object Storage*. 2023.
- [41] Amazon. *Amazon S3*. es. Page Version ID: 164024822. Dic. de 2024. URL: https://es.wikipedia.org/w/index.php?title=Amazon_S3&oldid=164024822 (visitado 28-03-2025).
- [42] MongoDB, Inc. *MongoDB: The Developer Data Platform*. 2023.
- [43] Google. *Angular*. en. URL: <https://angular.dev/> (visitado 30-08-2025).
- [44] Tiangolo. *FastAPI*. en. URL: <https://fastapi.tiangolo.com/> (visitado 30-08-2025).
- [45] Mingxing Tan y Quoc V. Le. *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. arXiv:1905.11946 [cs]. Sep. de 2020. DOI: [10.48550/arXiv.1905.11946](https://arxiv.org/abs/1905.11946). URL: <http://arxiv.org/abs/1905.11946> (visitado 27-03-2025).
- [46] Glenn Jocher y Jing Qiu. *Ultralytics YOLO11*. Ver. 11.0.0. 2024. URL: <https://github.com/ultralytics/ultralytics>.
- [47] Wikipedia. *Curva ROC*. es. Page Version ID: 166160029. Mar. de 2025. URL: https://es.wikipedia.org/w/index.php?title=Curva_ROC&oldid=166160029 (visitado 01-04-2025).
- [48] M. Soni y S. Soni. *Software Engineering Text Book*. 2024. URL: https://books.google.com.uy/books?id=L_YxEQAAQBAJ.
- [49] Wikipedia. *QGIS*. es. Page Version ID: 166170815. Mar. de 2025. URL: <https://es.wikipedia.org/w/index.php?title=QGIS&oldid=166170815> (visitado 09-04-2025).
- [50] Matei A. Zaharia et al. «Accelerating the Machine Learning Lifecycle with MLflow». En: *IEEE Data Eng. Bull.* 41 (2018), págs. 39-45. URL: <https://api.semanticscholar.org/CorpusID:83459546>.
- [51] WSO2. *Deliver Seamless Login Experiences with WSO2 Identity Server*. URL: <https://wso2.com/identity-server/>, <https://wso2.com/identity-server/> (visitado 04-04-2025).
- [52] Restic. *restic · Backups done right!* URL: <https://restic.net/> (visitado 09-04-2025).

- [53] Bacula. *Documentation*. en-US. URL: <https://www.bacula.org/documentation/> (visitado 04-04-2025).
- [54] Amazon Web Services. *AWS | Almacenamiento de datos seguro en la nube (S3)*. es-ES. URL: <https://aws.amazon.com/es/s3/> (visitado 04-04-2025).
- [55] LTB Project. *LDAP Tool Box project documentation — LDAP Tool Box project 1.0 documentation*. URL: <https://ltb-project.org/documentation/index.html#> (visitado 04-09-2025).
- [56] Wikipedia. *VirtualBox*. es. Page Version ID: 166376510. Mar. de 2025. URL: <https://es.wikipedia.org/w/index.php?title=VirtualBox&oldid=166376510> (visitado 04-04-2025).
- [57] Josh Aas et al. «Let's Encrypt: An Automated Certificate Authority to Encrypt the Entire Web». En: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*. CCS '19. New York, NY, USA: Association for Computing Machinery, nov. de 2019, págs. 2473-2487. ISBN: 978-1-4503-6747-9. DOI: 10.1145/3319535.3363192. URL: <https://dl.acm.org/doi/10.1145/3319535.3363192> (visitado 04-09-2025).
- [58] Ngrok, Inc. *ngrok | API Gateway, Kubernetes Ingress, Webhook Gateway*. en. URL: <https://ngrok.com/> (visitado 04-09-2025).
- [59] Cloudflare. *Conecta, protege y desarrolla en cualquier parte*. es-es. URL: <https://www.cloudflare.com/es-es/> (visitado 04-09-2025).
- [60] Traefik Labs. *Traefik, The Cloud Native Application Proxy*. en. URL: <https://traefik.io/traefik> (visitado 04-09-2025).
- [61] Bruno Masoller. *brunomaso1/uba-ceia at ceia-proy-final*. URL: <https://github.com/brunomaso1/uba-ceia/tree/ceia-proy-final> (visitado 04-04-2025).
- [62] Wikipedia. *Entorno de desarrollo integrado*. es. Page Version ID: 165982855. Mar. de 2025. URL: https://es.wikipedia.org/w/index.php?title=Entorno_de_desarrollo_integrado&oldid=165982855 (visitado 09-04-2025).
- [63] DrivenData. *Cookiecutter Data Science*. URL: <https://cookiecutter-data-science.drivendata.org/> (visitado 04-09-2025).
- [64] Wikipedia. *Talón de prueba*. es. Page Version ID: 164505596. Ene. de 2025. URL: https://es.wikipedia.org/w/index.php?title=Tal%C3%B3n_de_prueba&oldid=164505596 (visitado 04-04-2025).
- [65] Wikipedia. *Objeto simulado*. es. Page Version ID: 157309935. Ene. de 2024. URL: https://es.wikipedia.org/w/index.php?title=Objeto_simulado&oldid=157309935 (visitado 04-04-2025).
- [66] Wikipedia. *Lead time*. en. Page Version ID: 1279081155. Mar. de 2025. URL: https://en.wikipedia.org/w/index.php?title=Lead_time&oldid=1279081155 (visitado 09-04-2025).
- [67] Progress Chef's Bento. *bento/ubuntu-24.04*. URL: <https://portal.cloud.hashicorp.com/vagrant/discover/bento/ubuntu-24.04> (visitado 04-04-2025).
- [68] HashiCorp. *HashiCorp Cloud Platform | Bento*. URL: <https://portal.cloud.hashicorp.com/vagrant/discover/bento> (visitado 04-04-2025).
- [69] meaghanlewis. *Información general sobre la tecnología Hyper-V*. es-es. Ene. de 2025. URL: <https://learn.microsoft.com/es-es/windows-server/virtualization/hyper-v/hyper-v-overview> (visitado 04-04-2025).
- [70] Wikipedia. *VMware*. es. Page Version ID: 165895175. Mar. de 2025. URL: <https://es.wikipedia.org/w/index.php?title=VMware&oldid=165895175> (visitado 04-04-2025).
- [71] Jeff McKenna et al. *MapServer*. Ene. de 2025. DOI: 10.5281/ZENODO.6994443. URL: <https://zenodo.org/doi/10.5281/zenodo.6994443> (visitado 05-09-2025).

- [72] Leonard Richardson. «Beautiful soup documentation». En: *April* (2007).
- [73] Fatih Cagatay Akyon, Sinan Onur Altinuc y Alptekin Temizel. «Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection». En: *2022 IEEE International Conference on Image Processing (ICIP)* (2022), págs. 966-970. DOI: [10.1109/ICIP46576.2022.9897990](https://doi.org/10.1109/ICIP46576.2022.9897990). URL: <https://ieeexplore.ieee.org/document/9897990>.
- [74] *Oversampling and undersampling in data analysis*. en. Page Version ID: 1305290838. Ago. de 2025. URL: https://en.wikipedia.org/w/index.php?title=Oversampling_and_undersampling_in_data_analysis&oldid=1305290838 (visitado 05-09-2025).
- [75] Wei Hua y Qili Chen. «A survey of small object detection based on deep learning in aerial images». en. En: *Artificial Intelligence Review* 58.6 (mar. de 2025), pág. 162. ISSN: 1573-7462. DOI: [10.1007/s10462-025-11150-9](https://doi.org/10.1007/s10462-025-11150-9). URL: <https://doi.org/10.1007/s10462-025-11150-9> (visitado 20-09-2025).
- [76] Tsung-Yi Lin et al. «Microsoft COCO: Common Objects in Context». En: *CoRR* abs/1405.0312 (2014). _eprint: 1405.0312. URL: <http://arxiv.org/abs/1405.0312>.
- [77] Jacob Murel Ph.D. y Eda Kavlakoglu. *What is transfer learning?* | IBM. en. Feb. de 2024. URL: <https://www.ibm.com/think/topics/transfer-learning> (visitado 06-09-2025).
- [78] Jason Yosinski et al. *How transferable are features in deep neural networks?* arXiv:1411.1792 [cs]. Nov. de 2014. DOI: [10.48550/arXiv.1411.1792](https://doi.org/10.48550/arXiv.1411.1792). URL: <http://arxiv.org/abs/1411.1792> (visitado 05-09-2025).
- [79] Ian Goodfellow, Yoshua Bengio y Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [80] Alhassan Mumuni y Fuseini Mumuni. «Data augmentation: A comprehensive survey of modern approaches». En: *Array* 16 (dic. de 2022), pág. 100258. ISSN: 2590-0056. DOI: [10.1016/j.array.2022.100258](https://doi.org/10.1016/j.array.2022.100258). URL: <https://www.sciencedirect.com/science/article/pii/S2590005622000911> (visitado 06-09-2025).
- [81] Ultralytics. *Hyperparameter Tuning*. en. URL: <https://docs.ultralytics.com/guides/hyperparameter-tuning> (visitado 08-09-2025).
- [82] Internet Engineering Task Force. *GeoJSON*. URL: <https://geojson.org/> (visitado 20-09-2025).
- [83] Volodymyr Agafonkin. *Leaflet, a JavaScript library for interactive maps*. URL: <https://github.com/Leaflet/Leaflet>.
- [84] Roboflow. *Supervision*. URL: <https://github.com/roboflow/supervision>.
- [85] Jan Hosang, Rodrigo Benenson y Bernt Schiele. *Learning non-maximum suppression*. arXiv:1705.02950 [cs]. Mayo de 2017. DOI: [10.48550/arXiv.1705.02950](https://doi.org/10.48550/arXiv.1705.02950). URL: <http://arxiv.org/abs/1705.02950> (visitado 07-11-2025).
- [86] Jun Chu et al. «Syncretic-NMS: A Merging Non-Maximum Suppression Algorithm for Instance Segmentation». En: *IEEE Access* 8 (2020), págs. 114705-114714. ISSN: 2169-3536. DOI: [10.1109/ACCESS.2020.3003917](https://doi.org/10.1109/ACCESS.2020.3003917). URL: <https://ieeexplore.ieee.org/document/9121955> (visitado 08-11-2025).